Week 5 — Designing a Simple RISC-V CPU

Format: seven daily sections collected into one weekly mastery module

Suggested workload: approximately 21 focused hours, adjustable to mastery

> The ISA defines the machine software may rely on. A microarchitecture must make that machine appear while real values move through finite, timed hardware.

This module turns the Week 4 hardware foundation into a small RISC-V CPU model. It begins with an explicit RV32I subset and architectural state, derives the ALU, register file, PC, decoder, and single-cycle paths, executes a hand-encoded loop, then introduces pipelining, hazards, caches, and address translation. The purpose is systems understanding: the learner should be able to connect compiler output, saved trap state, page faults, PMU evidence, QEMU/KVM behavior, and eventually ToyOS to concrete CPU mechanisms.

This is not an HDL-specialization module and does not claim to implement a production RISC-V platform. “Week” names a mastery package, not a calendar deadline. Repeat a trace or split a lab across sessions whenever the architectural result, implementation mechanism, or ownership of a decision is unclear.

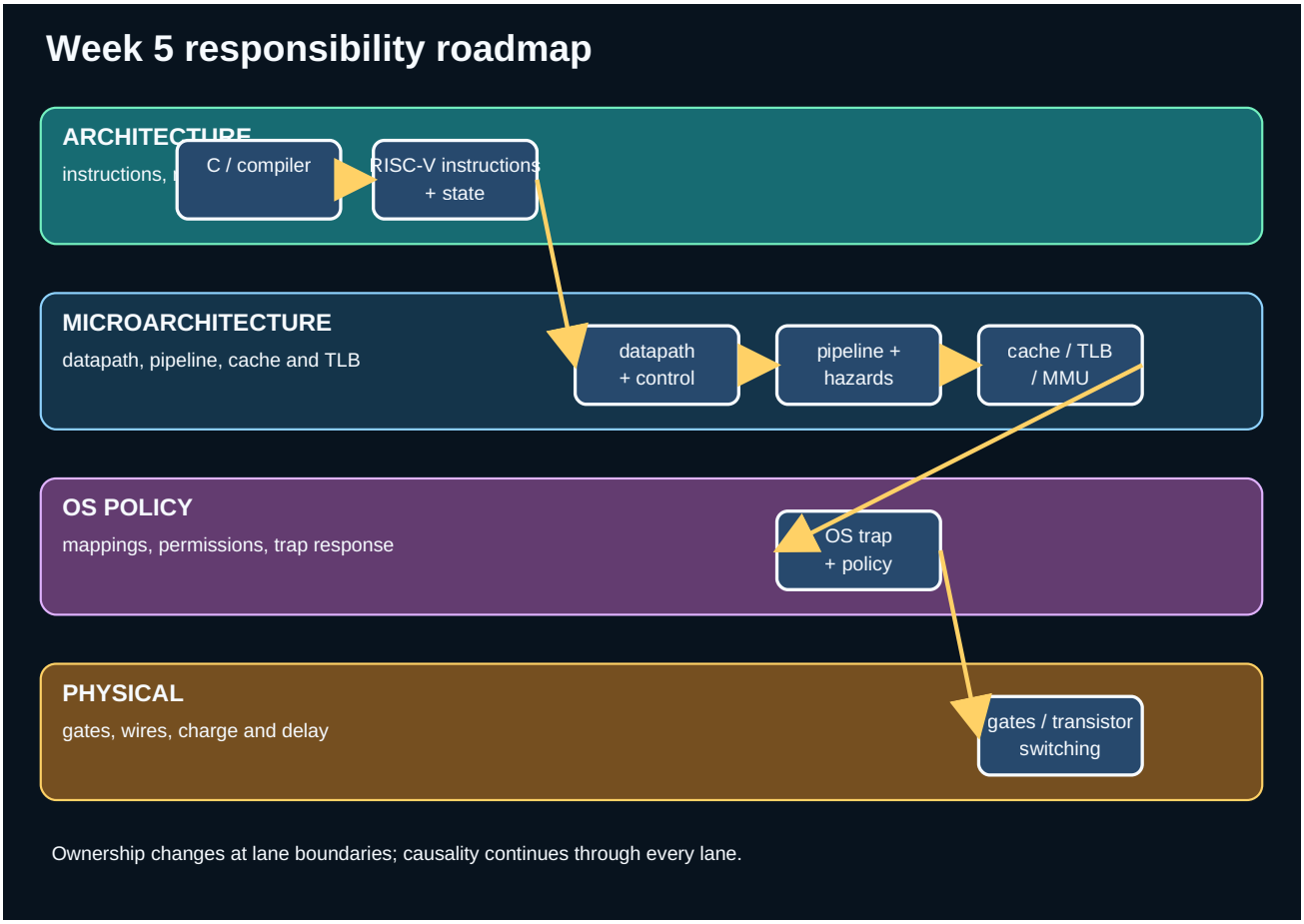
The responsibility map

Keep four layers separate throughout the module:

- **Architectural:** ISA-visible registers, instruction effects, addresses, ordering guarantees, and exceptions.
- **Microarchitectural:** datapaths, controls, pipeline registers, forwarding, predictors, caches, TLBs, stalls, fills, and replacement choices.
- **OS policy:** mappings, page-table entries, permissions, fault handling, scheduling, and process isolation.
- **Physical:** gates, wires, clocked state, voltage ranges, capacitance, delay, energy, and transistor switching.

The layers form a causal chain, but they do not own the same decisions.

```
C and compiler
  → RISC-V architectural instructions
  → datapath and control
  → pipelined implementation
  → cache/TLB/MMU interactions
  → OS-configured translation and trap response
  → gates and physical switching
```



The same computation crosses four ownership lanes while preserving a single causal chain.

Daily roadmap and exact files

Day 1 — An RV32I Subset and Its Datapath Contract

Choose a documented RV32I teaching subset, separate official ISA behavior from implementation choices, decode the required formats, name architectural state, and derive the datapath/control contract.

Lesson:

```
05-riscv-cpu-microarchitecture/sections/day-001-rv32i-subset-and-datapath-contract.md
```

Challenge:

```
05-riscv-cpu-microarchitecture/challenges/day-001-rv32i-subset-trace.py
```

Evidence: decode R/I/S/B/U/J examples, trace one complete architectural transition, reject unsupported words safely, and label every claim as ISA or teaching-implementation behavior. Keep the model's instruction/data request ports distinct from the SRAM arrays, cache hierarchy, memory controller, and DRAM banks that may implement those requests.

Day 2 — ALU, Register File, and Program Counter

Make modulo arithmetic, signed/unsigned comparison, two-read/one-write register state, the `x0` invariant, and next-PC selection precise. Observe combinational values separately from edge-triggered state.

Lesson:

```
05-riscv-cpu-microarchitecture/sections/day-002-alu-register-file-and-program-counter.md
```

Challenges:

```
05-riscv-cpu-microarchitecture/challenges/day-002-alu-register-file.sv
05-riscv-cpu-microarchitecture/challenges/day-002-alu-register-file-tb.sv
05-riscv-cpu-microarchitecture/challenges/day-002-run-alu-register-file.sh
```

Evidence: preserve clean lint/simulation output, compare `SLT` with `SLTU`, demonstrate that writes to `x0` have no effect, and explain current-PC state versus next-PC candidates. Treat the RTL register array as one implementation model: physical register files may use custom cells, banking, replication, or synthesized flops, and their ports need not map one-for-one to the source-level array.

Day 3 — Decoder, Control, and Single-Cycle Paths

Turn instruction bits into immediate construction, legality checks, mux selections, ALU operation, write enables, memory requests, and next-PC control. Derive why the single-cycle load path can determine the clock.

Lesson:

```
05-riscv-cpu-microarchitecture/sections/day-003-decoder-control-and-single-cycle-paths.md
```

Challenge:

```
05-riscv-cpu-microarchitecture/challenges/day-003-control-decoder.py
```

Evidence: inspect legal arithmetic/load/store/branch controls, reconstruct a negative branch immediate, prove illegal encodings disable architectural writes, and draw the load critical path. Carry the request forward to the stallable cache and lower-memory path rather than assuming every physical memory responds combinationally.

Day 4 — Integrating and Running a Tiny RV32I Core

Connect fetch, decode, register reads, ALU/address work, memory, writeback, and next-PC selection around an explicit commit boundary. Run a hand-encoded loop that sums signed words, stores the result, and loads it back.

Lesson:

```
05-riscv-cpu-microarchitecture/sections/day-004-integrating-and-running-a-tiny-rv32i-core.md
```

Challenge:

```
05-riscv-cpu-microarchitecture/challenges/day-004-tiny-rv32i-core.py
```

Evidence: predict all 25 model cycles, annotate one loop iteration as state → candidates → commit, verify final registers and memory, and explain why model errors are not yet architectural traps.

Day 5 — Pipelining and Throughput

Split the long combinational path into IF, ID, EX, MEM, and WB stages. Distinguish latency from throughput, account for register overhead and stage imbalance, and carry identity/control/validity with each instruction.

Lesson:

```
05-riscv-cpu-microarchitecture/sections/day-005-pipelining-and-throughput.md
```

Challenge:

```
05-riscv-cpu-microarchitecture/challenges/day-005-pipelining-and-throughput.py
```

Evidence: derive the clock period and `k + N - 1` cycle count, inspect an ideal pipeline trace, calculate finite-batch speedup, and separate pipeline state from architectural retirement.

Day 6 — Hazards, Forwarding, Stalls, and Branches

Classify structural, data, and control hazards. Use forwarding when a result exists, stall when it does not, and flush wrong-path work after branch resolution while preserving precise architectural state.

Lesson:

```
05-riscv-cpu-microarchitecture/sections/day-006-hazards-forwarding-stalls-and-branches.md
```

Challenge:

```
05-riscv-cpu-microarchitecture/challenges/day-006-hazards-forwarding-stalls-and-branches.py
```

Evidence: preserve traces for an ALU bypass, load-use interlock, no-forwarding wait, unified-memory structural conflict, branch misprediction, and correct prediction; explain the architectural error each mechanism prevents.

Day 7 — Cache, TLB, MMU, and End-to-End Task Trace

Connect instruction and data requests to caches, miss backpressure, address translation, a TLB, page-table walks, permissions, and precise faults. Keep memory ordering and coherence at overview depth, then trace a C `sum` from compiler-selected RISC-V instructions down to gates and transistor switching.

Lesson:

```
05-riscv-cpu-microarchitecture/sections/day-007-cache-tlb-mmu-and-end-to-end-task-trace.md
```

Challenge:

```
05-riscv-cpu-microarchitecture/challenges/day-007-cache-tlb-trace.py
```

Evidence: annotate TLB and cache hit/miss paths, observe a page-table walk, prove that a read-only PTE blocks a store even when the line is cached, distinguish absent mapping from denied permission, and assign every decision to architecture, microarchitecture, OS policy, or physics. Follow one miss below the cache through SRAM tag/data lookup, line fill, a memory controller, and a bounded DRAM row operation before tracing the response back to the waiting load.

Adjustable workload guide

The nominal budget is approximately 21 focused hours:

```
Day 1   3 h   RV32I subset, architectural state, formats, datapath contract
Day 2   3 h   ALU, comparison, register file, x0, PC state
Day 3   3 h   decode, legality, controls, single-cycle paths
Day 4   3 h   integrated core and complete loop execution
Day 5   3 h   pipeline timing, latency, throughput, retirement
Day 6   3 h   hazards, forwarding, stalls, prediction, flush
Day 7   3 h   caches, TLB/MMU, faults, end-to-end trace
```

This is an estimate, not a completion rule. Spend longer where a prediction and trace disagree. Familiar syntax may be reviewed quickly, but do not skip the explanation of architectural effects, timing assumptions, or model limits.

Integrated weekly evidence

Explain

Without notes:

1. separate official ISA behavior, the chosen subset, and implementation;
2. derive the data and control path for `ADD`, `LW`, `SW`, a branch, and an indirect control transfer;
3. explain why combinational candidates are not committed state;
4. distinguish latency, throughput, clock period, CPI, and retirement;
5. explain when forwarding works and why a load-use pair can still stall;
6. explain how prediction and flushing preserve the selected instruction stream;
7. distinguish cache miss, TLB miss, invalid translation, and permission fault;
8. trace one result down through gates to physical charge movement.

Draw

Produce one connected drawing with:

- PC, instruction fields, immediate generation, register file, ALU, memory, writeback, and next-PC paths;
- IF/ID, ID/EX, EX/MEM, and MEM/WB state with valid/kill metadata;
- forwarding sources, load-use detection, and branch redirect;
- instruction/data cache paths and miss backpressure;

- virtual page number, page offset, TLB, page-table walker, permission check, physical address, and fault path;
- one selected datapath bit realized by gates and transistor-controlled charging/discharging.

Mark architectural, microarchitectural, OS-policy, and physical elements using distinct labels.

Observe

Preserve a compact evidence bundle:

- Day 1 decode/state trace and unsupported-word diagnostic;
- Day 2 RTL lint/simulation result and signed/unsigned comparison;
- Day 3 legal/illegal control bundles;
- Day 4 complete loop trace and final state;
- Day 5 timing derivation and ideal pipeline trace;
- Day 6 forwarding, stall, structural-conflict, and flush traces;
- Day 7 cache/TLB trace and all self-test PASS lines;
- one statement per lab describing what its model cannot prove.

Correlate existing evidence end to end

Do not create a new lab. Reuse the existing artifacts as a cross-layer workflow:

1. Start with the Week 3 source-to-object evidence and ELF loading evidence. `readelf` proves container metadata for the inspected file; `objdump` proves how that tool decodes that file's bytes for its declared machine. Host-architecture output is not RISC-V CPU evidence.
2. Select one operation in the C source and correlate it with the compiler-selected instructions only when source locations, symbols, and the actual disassembly support the match. Then use the Week 5 CPU and cache traces to demonstrate the chosen teaching model's architectural updates, stalls, hits, misses, and fills.
3. Reuse the Week 4 RTL and waveform evidence and Yosys synthesis evidence. Simulation proves tested RTL behavior; synthesis proves the selected RTL was transformed into a netlist under the recorded script and cell model. Neither proves that the Week 5 Python core is that RTL.
4. Reuse the Week 4 CMOS/ngspice evidence and delay/power evidence to observe voltage, current, delay, and charging in the simulated circuit. That proves behavior of the stated device models and stimulus, not the transistor implementation or energy of a fabricated RISC-V core.

The result is an evidence-backed causal chain, not a claim that these separate teaching artifacts form one formally equivalent implementation.

Build

Complete one bounded extension in each model, or build one integrated explanation that:

1. begins with a 32-bit instruction and old architectural state;
2. derives operands, controls, and candidate results;
3. commits only enabled effects;
4. follows the same dependencies through a pipeline;
5. inserts a justified bypass, stall, or flush;

6. waits safely for cache/TLB misses;
7. reports a denied or absent mapping without committing the access;
8. ends with layer-labelled physical switching.

Use the recurring loop:

```
predict → run → inspect → explain the responsible layer
        → change one condition → rerun → compare
```

Integrated mastery checkpoint

Trace this task end to end:

```
int sum(const int *a, int n) {
    int s = 0;
    for (int i = 0; i < n; ++i)
        s += a[i];
    return s;
}
```

A complete answer must connect:

```
C abstract-machine behavior
  → compiler-selected RISC-V instructions
  → architectural PC/register/memory effects
  → fetch, decode, execute, memory, and writeback paths
  → dependencies, forwarding, stalls, branch redirect, and retirement
  → instruction/data cache hits, misses, fills, and backpressure
  → TLB hit or hardware page-table walk
  → OS-created mapping and permission data
  → precise fault and OS response when an access cannot complete
  → mux/decoder/adder/register gates
  → transistor conduction, capacitance, delay, and switching energy
```

For every arrow, identify whether it is an architectural requirement, a microarchitectural choice, OS policy, or a physical mechanism. The explanation must remain correct when a load hits, misses, waits for translation, or faults.

Bridge to ToyOS

The CPU provides mechanisms: architectural state, privilege transitions, precise traps, interrupt delivery, translation, permissions, and memory access. ToyOS will provide policy: boot-time setup, physical-page allocation, page-table construction, mappings, trap handling, task state, scheduling, user mode, and system calls.

```
simple CPU model
  → architectural traps and privilege
  → ToyOS boot and exception entry
  → physical-page allocation and virtual memory
  → interrupts, tasks, user mode, and system calls
```

Do not auto-fill confidence, completion, hours, or mastery. Those remain the learner's evidence-based assessment.

Day 1 — An RV32I Subset and Its Datapath Contract

Target time: approximately 2–3 hours

- Choose the architectural contract: 35–45 minutes
- Decode the required formats: 40–50 minutes
- Derive datapath and control requirements: 45–55 minutes
- Run and explain the Python trace lab: 30–40 minutes

> A program says `add x5, x6, x7`. What must be true after that instruction retires, and which parts of the machine are free to differ?

Why this day exists

Systems software depends on an architectural machine: registers, addresses, instruction effects, and precisely defined control flow. A CPU designer must build some implementation that preserves that contract. Those are related objects, but they are not the same object.

```
official RISC-V ISA
        ↓ choose a documented teaching subset
architectural state transition
        ↓ derive required data movement
datapath + control contract
        ↓ choose one implementation
single-cycle teaching CPU
```

Today stops at that implementation contract. We will not begin with HDL and hope a CPU emerges. First we decide exactly which instructions exist, what state they can change, which values each datapath must produce, and what this small design deliberately does not implement.

This is necessary CPU-design depth for systems programming. It is not an HDL specialization. The payoff is being able to look at a faulting instruction, kernel register frame, emulator trace, or disassembly and ask: which architectural transition was requested, and what machinery had to realize it?

By the end, you should be able to:

- distinguish official RV32I behavior from our subset and implementation;
- name the architectural state visible to this model;
- decode the R, I, S, B, U, and J formats used here;
- derive register-file, ALU, immediate, memory, and next-PC requirements;
- state the assumptions that make a single-cycle diagram possible;
- identify deferred instructions and exception behavior without pretending they do not exist;
- trace instruction bits into control decisions and an architectural update.

---

1. Three contracts, not one

Keep these statements separate:

1. **The RISC-V unprivileged ISA specification** defines instruction encodings and architectural behavior.
2. **Our pedagogical subset** chooses a finite set of those instructions and narrows the execution environment.
3. **Our single-cycle implementation** chooses one way to realize the subset.

For example, the official ISA says what `ADD` computes. It does not require one physical adder, one clock cycle, a single-cycle datapath, combinational memory, or any particular SystemVerilog module boundary.

```
ISA-visible fact:
    ADD writes (rs1 + rs2) modulo 2^32 to rd

implementation choices:
    ripple or carry-lookahead adder?
    shared address/ALU hardware or separate adders?
    one cycle or several?
    pipelined?
    physical register renaming?
```

All conforming implementations must produce the required architectural result. Their internal paths, timing, and speculative state may differ.

Why RV32I?

`RV32I` has 32-bit integer registers and a 32-bit base integer address space. It gives us regular 32-bit instruction forms, explicit register arithmetic, loads/stores, and PC-relative control flow without requiring multiplication, floating point, vectors, atomics, or compressed instructions.

That does not make this exact teaching CPU a practical Linux platform. A real RISC-V Linux system needs privilege, exceptions, interrupts, address translation, atomics and other platform facilities beyond this subset. RV32I is the clean ISA boundary on which to learn the datapath.

Architecture comparison — decode is a contract, not a ranking

RV32I is especially convenient for this teaching datapath because base instructions are fixed-width, arithmetic is register-to-register, and memory is accessed through explicit loads and stores. A64 instructions are also fixed 32-bit words and use a load/store model. x86-64 instead has variable-length instructions and permits many register/memory forms, so its architectural decode rules are denser.

Those are **ISA** differences, not performance rankings or required internal pipelines. An x86-64 core may decode instructions into internal micro-operations; an AArch64 or RISC-V core may also crack, fuse, or otherwise transform instructions. Those choices are **microarchitecture** so long as retired behavior preserves the ISA. The subset and single-cycle organization used here are teaching choices, not consequences of “RISC.” See Architecture Comparison for Systems Programmers.

2. The chosen subset

The following list is the complete instruction contract for the CPU we will derive:

```
register ALU     ADD SUB AND OR XOR SLT SLTU
immediate ALU    ADDI ANDI ORI XORI SLTI SLTIU
memory           LW SW
branches         BEQ BNE BLT BGE BLTU BGEU
upper immediate  LUI AUIPC
control transfer JAL JALR
```

This is not a newly invented ISA. Every listed instruction is an official RV32I instruction. The selection is ours.

Why these instructions?

The subset is small enough to trace but broad enough to expose the important datapath choices:

- `ADD` / `ADDI` need ordinary arithmetic and occur constantly in compiled code.
- `SUB` and comparisons force us to distinguish arithmetic results from branch decisions.
- bitwise operations implement masks used in kernels and drivers.
- signed and unsigned comparisons show that bits carry no permanent type.
- `LW` / `SW` create the register–address–memory boundary.
- branches select between `pc+4` and `pc+imm`.
- `LUI` and `AUIPC` help construct constants and PC-relative addresses.
- `JAL` and `JALR` expose calls, returns, link values, and indirect control.

`JALR` computes `(rs1 + sign_extend(imm)) & ~1` as its target. Clearing bit zero is official behavior, not a convenience invented by our datapath.

Explicitly outside the subset

```
byte/halfword loads and stores
shifts
FENCE and memory-ordering machinery
ECALL and EBREAK
CSRs, privilege levels, traps, and trap return
interrupts
M multiplication/division
A atomics
F/D floating point
V vectors
C compressed instructions
```

These omissions matter. Without `ECALL`, privilege, and trap state, this CPU cannot enter a kernel through the normal system-call path. Without atomics and a memory-ordering implementation it cannot support realistic multi-hart kernel synchronization. We defer those mechanisms; we do not redefine them as no-ops.

---

3. Architectural state

For this subset, the programmer-visible state is:

```
pc               address of the current instruction
x0..x31          32 integer registers, each 32 bits
memory bytes      instruction and data contents addressed by the program
```

`x0` always reads as zero. An attempted write to `x0` has no architectural effect. The other registers have no guaranteed ISA-defined reset values.

The architectural transition for one successfully completed instruction can be written:

```
(old pc, old registers, old memory)
                ↓ instruction semantics
(new pc, new registers, new memory)
```

Most instructions change one integer register and the PC. A store changes memory and the PC. A branch changes only the PC. A write with `rd=x0` changes no retained integer register.

Architectural state is not every internal bit

An implementation may have:

- an instruction register;
- control flops;
- pipeline registers;
- cache tags;
- branch-predictor tables;
- physical registers;
- outstanding memory-request state.

Those are microarchitectural state. Software does not get to demand their particular layout merely because they exist. Conversely, a context switch or virtual CPU must preserve the architectural state relevant to the enabled ISA and privilege extensions.

A precise observation boundary

Our teaching model treats an instruction as completing at the active clock edge:

```
before edge   old architectural state drives combinational paths
at edge       register file, PC, or data memory accepts selected updates
after edge    the next instruction observes the new architectural state
```

This makes the state transition visible. It is not a claim that the official ISA defines an instruction as one clock cycle.

---

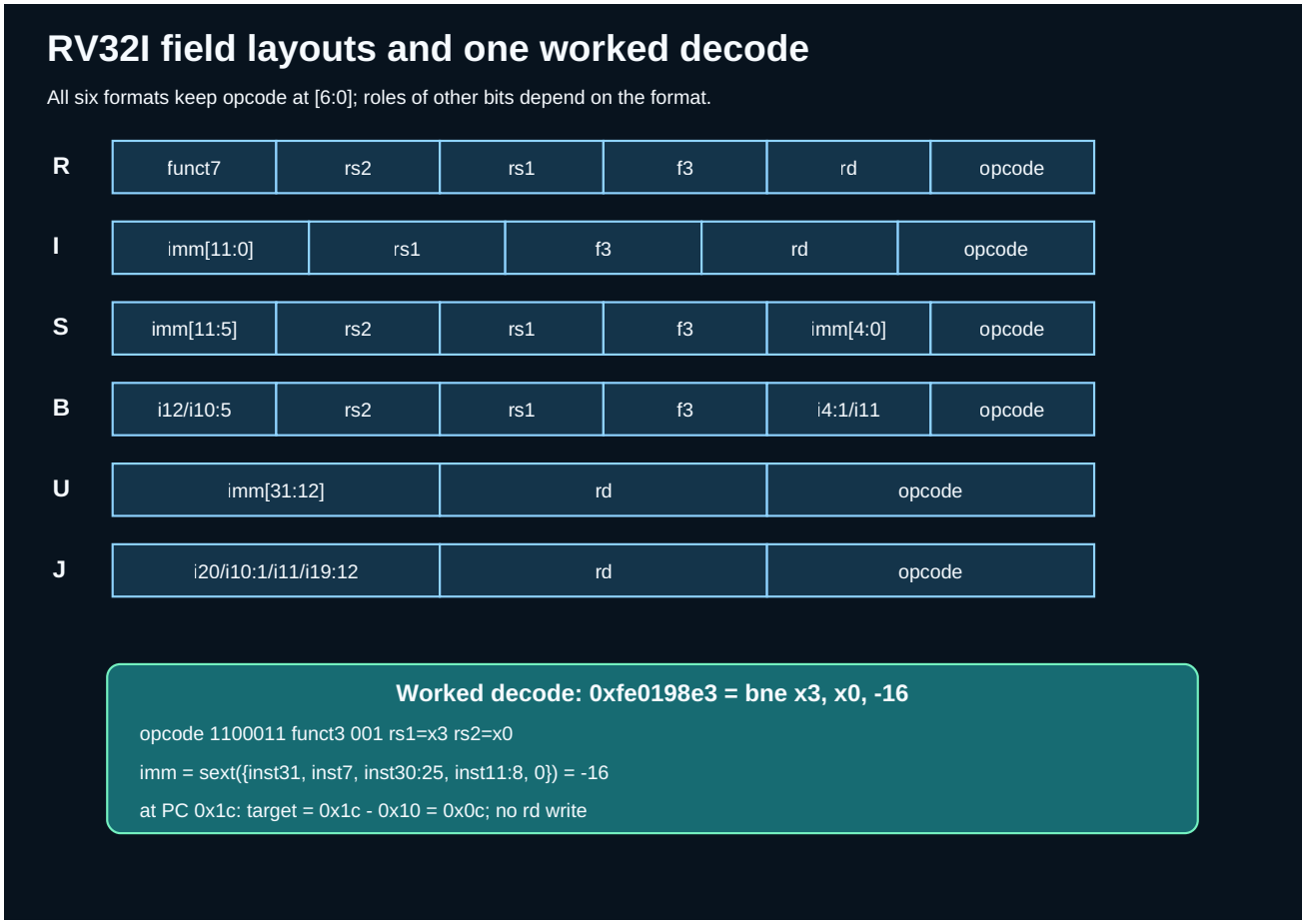
4. Six formats supply the needed fields

All instructions in this teaching subset are 32 bits. Because the `C` extension is absent, valid instruction addresses are four-byte aligned for this implementation.

Common field positions are:

```
opcode  instruction[6:0]
rd      instruction[11:7]
funct3  instruction[14:12]
rs1     instruction[19:15]
rs2     instruction[24:20]
funct7  instruction[31:25]
```

The format determines which fields have meaning.



Stable field positions support six formats; decode determines each slice's role.

R-type — register ALU

```
31       25 24    20 19    15 14  12 11     7 6       0
+----------+--------+--------+------+---------+---------+
| funct7   | rs2    | rs1    |funct3| rd      | opcode  |
+----------+--------+--------+------+---------+---------+
```

`ADD`, `SUB`, `AND`, `OR`, `XOR`, `SLT`, and `SLTU` read two registers and write one. `opcode`, `funct3`, and sometimes `funct7` select the operation.

I-type — immediate ALU, load, and JALR

```
31                20 19    15 14  12 11     7 6       0
+-------------------+--------+------+---------+---------+
| imm[11:0]         | rs1    |funct3| rd      | opcode  |
+-------------------+--------+------+---------+---------+
```

The 12-bit immediate is sign-extended for this subset. The second ALU input comes from that immediate for `ADDI`, `LW`, and `JALR`, even though their final destinations differ.

S-type — store

```
31       25 24    20 19    15 14  12 11      7 6       0
+----------+--------+--------+------+----------+---------+
|imm[11:5] | rs2    | rs1    |funct3| imm[4:0] | opcode  |
+----------+--------+--------+------+----------+---------+
```

`SW` reads `rs1` as the base and `rs2` as the data. It has no `rd`, so those five bit positions help carry the address displacement.

B-type — conditional branch

```
imm[12]    = instruction[31]
imm[10:5]  = instruction[30:25]
imm[4:1]   = instruction[11:8]
imm[11]    = instruction[7]
imm[0]     = 0
```

The reconstructed 13-bit value is sign-extended and added to the current instruction's PC. Both source registers feed comparison logic. There is no `rd`.

U-type — upper immediate

```
31                                      12 11     7 6:0
+-----------------------------------------+---------+------+
| imm[31:12]                                | rd      |opcode|
+-----------------------------------------+---------+------+
```

`LUI` writes `imm[31:12] << 12`. `AUIPC` adds that same 32-bit immediate to the current PC.

J-type — JAL

```
imm[20]    = instruction[31]
imm[10:1]  = instruction[30:21]
imm[11]    = instruction[20]
imm[19:12] = instruction[19:12]
imm[0]     = 0
```

`JAL` writes `pc+4` to `rd` and selects `pc+imm` as the next PC. It therefore needs two results in the same instruction: a link value and a control-flow target.

Pause and derive

For each format, mark:

1. which registers are read;
2. whether a register is written;
3. how the immediate is reconstructed;
4. what can become the next PC;
5. whether memory is read or written.

That table is already most of a decoder specification.

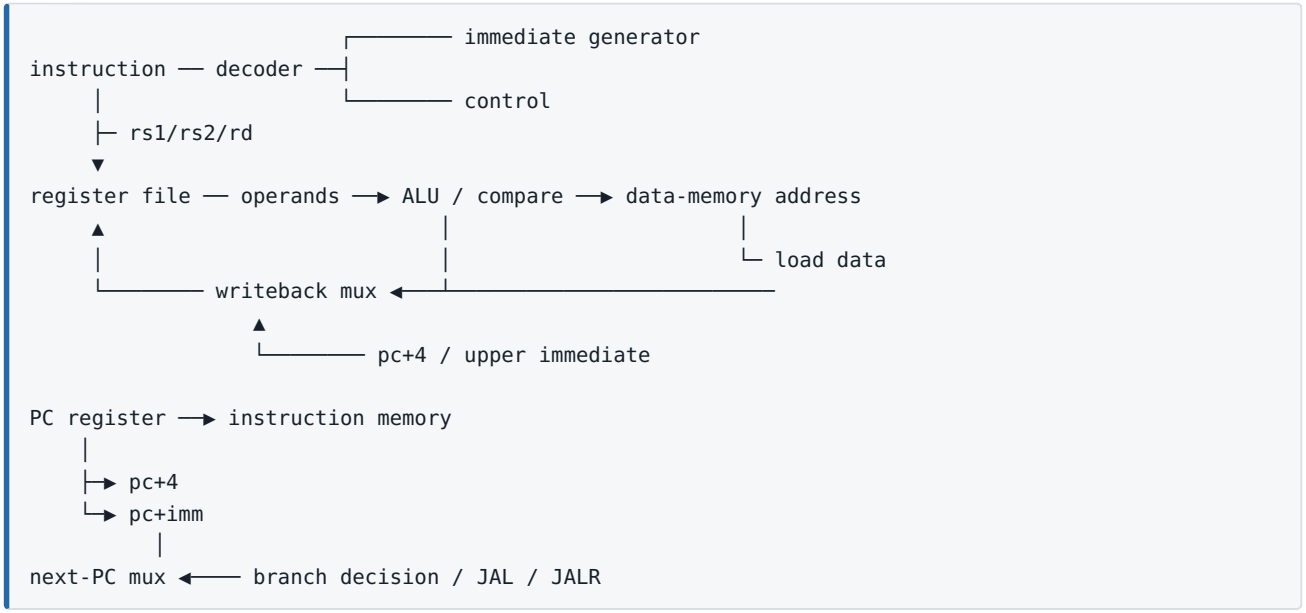
---

5. Derive the datapath from required values

Do not memorize a canonical CPU picture. Begin with values the instructions need:

```
current PC
instruction bits
rs1 value
rs2 value
decoded immediate
ALU result
memory load data
pc + 4
pc + immediate
rs1 + immediate, with bit 0 cleared for JALR target
```

A minimal block view is:



The drawing can use separate adders for `pc+4` and branch targets while the main ALU computes a data result. Sharing one adder would require multiple phases or a longer combinational arrangement. Separate adders are a single-cycle implementation choice, not an ISA rule.

Control contract

One useful controller interface is:

```
alu_op          ADD/SUB/AND/OR/XOR/SLT/SLTU/pass
alu_src_imm     choose rs2 or decoded immediate
imm_kind        I/S/B/U/J reconstruction
reg_write       permit rd update
writeback_sel   ALU / load data / pc+4 / U-immediate / pc+U-immediate
mem_read        request LW
mem_write       request SW
branch_kind     none/EQ/NE/LT/GE/LTU/GEU
next_pc_sel     sequential/branch/JAL/JALR
```

Exact signal names are implementation choices. The contract matters: illegal encodings must not accidentally write a register or memory.

Instruction-to-path table

Family	Register reads	ALU/compare use	Writeback	Next PC
R ALU	`rs1`, `rs2`	selected operation	ALU → `rd`	`pc+4`
I ALU	`rs1`	`rs1 op imm`	ALU → `rd`	`pc+4`
`LW`	`rs1`	address = `rs1+imm`	memory → `rd`	`pc+4`
`SW`	`rs1`, `rs2`	address = `rs1+imm`	memory write	`pc+4`
branch	`rs1`, `rs2`	selected compare	none	`pc+imm` or `pc+4`
`LUI`	none	upper immediate	immediate → `rd`	`pc+4`
`AUIPC`	none	`pc+upper_imm`	result → `rd`	`pc+4`
`JAL`	none	target = `pc+imm`	`pc+4` → `rd`	target
`JALR`	`rs1`	target = `(rs1+imm)&~1`	`pc+4` → `rd`	target

The table is an implementation review tool. If `SW` accidentally asserts `reg_write`, or a branch uses the wrong immediate kind, the architectural transition is wrong even if the circuit is electrically valid.

---

6. The single-cycle timing fiction—useful and explicit

Our first CPU assumes one long cycle:

```
PC output
 → instruction memory
 → decode and register-file read
 → immediate/ALU/compare
 → optional data-memory read
 → writeback selection
 → register-file and PC capture
```

To make that model executable, we assume:

- instruction memory supplies a word combinationally from the current PC;

- data memory supplies `LW` data combinationally;
- register-file reads are combinational;
- register-file writes occur at the active edge;
- `SW` memory update occurs at the active edge;
- the PC updates at the active edge;
- all combinational paths settle before that edge;
- instruction and data access are shown as distinct ports/memories;
- every implemented instruction completes in one cycle.

These assumptions are pedagogical, not ordinary high-performance memory interfaces. Real SRAMs, caches, buses, and DRAM have clocked protocols and variable latency. A single-cycle CPU's period must accommodate its longest path, often a load:

```
PC → instruction fetch → decode/register read
   → address ALU → data memory → writeback mux → register setup
```

Even a simple `ADD` waits for that same global period. The inefficiency motivates multicycle and pipelined designs later.

Harvard-looking ports do not redefine software memory

Separate instruction and data interfaces avoid two accesses competing for one port in the same cycle. This is a microarchitectural convenience. Whether the programmer sees one address space, whether code is writable, and how caches remain coherent are separate platform questions.

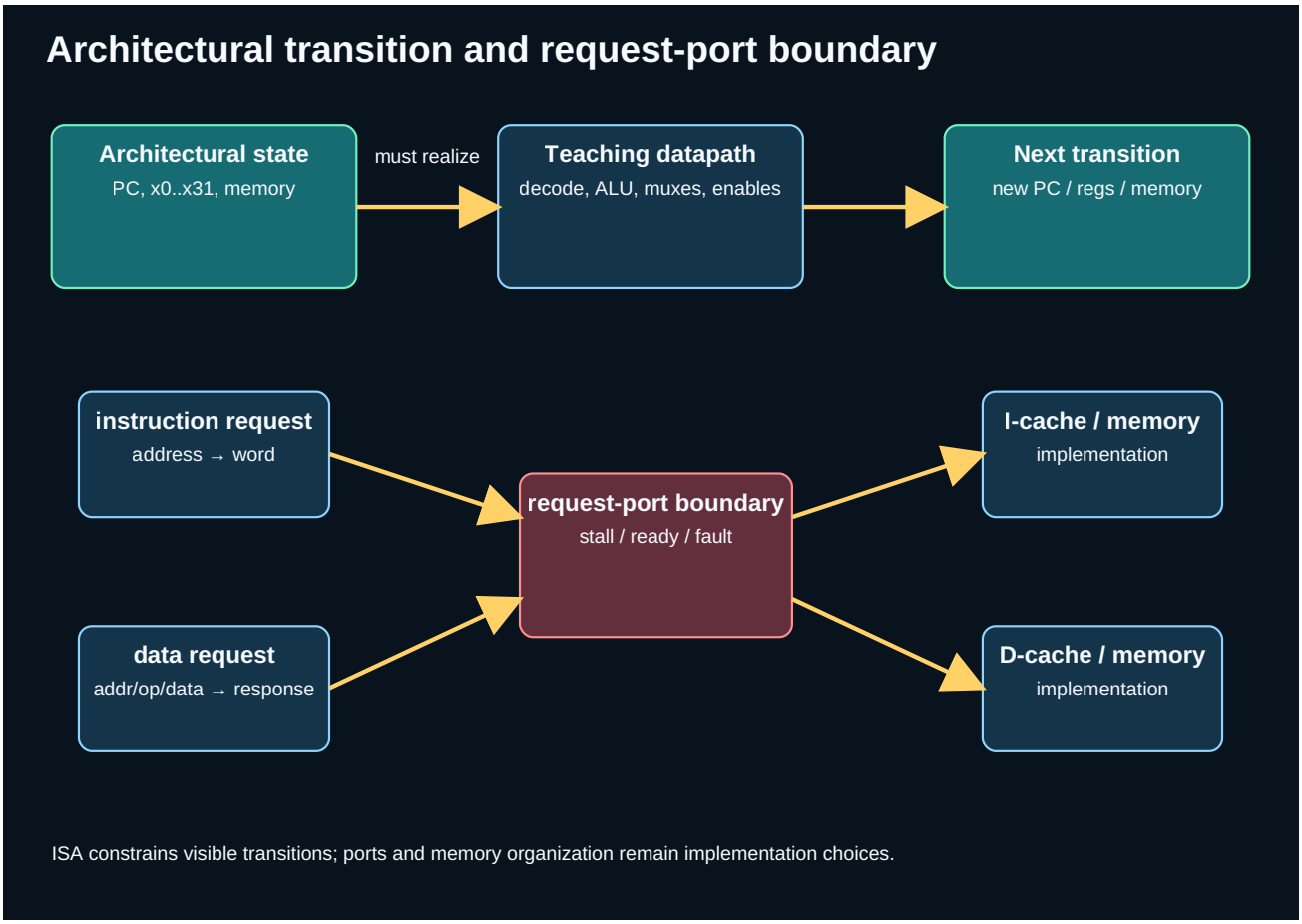
A memory port is not a memory array

For this lesson, “instruction memory” and “data memory” are bounded architectural-facing request abstractions: an address and operation eventually produce instruction bytes, load data, a completed store, or an exception. The ISA specifies the visible result of `LW` and `SW`; it does not specify these ports or require combinational response.

The implementation can translate one request through caches and controllers to very different physical storage:

```
teaching request port
  → cache/memory protocol and buffering
  → SRAM tag/data arrays on chip
  → memory controller and channel
  → bank/row/column operations in DRAM
```

An SRAM array contains addressed storage cells with wordlines, bitlines, and sense/write circuitry. Commodity DRAM is organized into banks and rows of capacitor-based cells with activation, sensing/restoration, precharge, and refresh requirements. Neither structure is implied by the source-level `memory[address]` notation, and neither normally promises this lesson's single-cycle combinational timing. Day 7 follows one cache miss through that lower path and back to the waiting load.



The ISA owns visible transitions; the teaching ports stop before the memory implementation.

---

7. Exceptions are deferred, not erased

Official RISC-V systems define exception behavior in conjunction with the ISA, privileged architecture, and execution environment. This teaching CPU does not yet contain trap CSRs, privilege levels, a trap vector, or restart logic.

We therefore impose a lab precondition:

```
fetched instruction is in the documented subset
PC is four-byte aligned
LW/SW effective address is four-byte aligned
instruction and data addresses are mapped by the lab
```

Outside that precondition, the Python model reports an error and stops. That is **lab behavior**, not the architectural definition of an illegal instruction or access fault.

Deferred cases include:

- illegal or unsupported instruction encoding;
- instruction-address misalignment;
- load/store address misalignment;
- instruction or data access fault;
- environment call and breakpoint;

- interrupts.

A later CPU must suppress inappropriate partial updates, record a cause and faulting PC, select a handler, and support the required return semantics. A kernel relies on that precision. Silently treating an unsupported opcode as `NOP` would corrupt the software-visible contract.

---

8. Python decode-and-trace laboratory

File:

```
05-riscv-cpu-microarchitecture/challenges/day-001-rv32i-subset-trace.py
```

No cross-compiler or third-party package is required.

From the repository root:

```
python3 05-riscv-cpu-microarchitecture/challenges/day-001-rv32i-subset-trace.py demo
python3 05-riscv-cpu-microarchitecture/challenges/day-001-rv32i-subset-trace.py trace
python3 05-riscv-cpu-microarchitecture/challenges/day-001-rv32i-subset-trace.py selftest
```

The trace prints, for each instruction:

```
instruction word → decoded fields → control/path summary
                 → old PC/register values → architectural update
```

Predict before running

The included loop sums four 32-bit words.

1. Which instruction family first reads data memory?
2. Which instruction writes data memory?
3. Which path produces the address used by `LW`?
4. Which comparison makes the loop branch?
5. Which value is fed back to the PC when the branch is taken?
6. Does the loop ever alter `x0`?

Then run only five steps:

```
python3 05-riscv-cpu-microarchitecture/challenges/day-001-rv32i-subset-trace.py trace --limit 5
```

Predict the sixth line before increasing the limit.

What the lab proves

It demonstrates that the selected encodings can be decoded and that the software model performs the documented state transitions for tested cases. It helps review immediate reconstruction and datapath selection.

It does not prove:

- cycle-accurate hardware behavior;
- RTL correctness;

- electrical timing;
- official compliance beyond the tested subset;
- exception, privilege, or memory-ordering behavior;
- that a physical CPU uses the same internal operations.

---

9. Systems and kernel relevance

This small datapath explains several larger mechanisms:

- A saved kernel trap frame names architectural registers and a PC because those are the state software must restore.
- A disassembler reconstructs fields much like our decoder, although it covers many more extensions and aliases.
- QEMU can model architectural transitions without copying a target CPU's physical datapath.
- KVM exposes virtual architectural state while host hardware supplies a very different microarchitecture.
- A page fault cannot simply write a load's `rd` and then report failure; the architecture needs a restartable, precise boundary.
- `JALR` underlies indirect calls and returns, so target formation becomes relevant to control-flow integrity and exploit analysis.
- `LW`/`SW` address generation is where virtual addressing, alignment, permissions, caches, and device mappings later attach.

The single-cycle machine is not realistic enough to run a kernel. It is simple enough to expose exactly where those mechanisms must connect.

---

10. Mastery evidence

Explain

Without notes:

- separate official ISA, subset, and implementation;
- justify every instruction family in the subset;
- name architectural state and give one microarchitectural counterexample;
- explain why `x0` needs both read and write behavior;
- derive each possible register writeback source;
- explain why the single-cycle load path tends to set the clock period;
- state what happens in this lab and what must eventually happen in hardware for an unsupported instruction.

Draw

Draw the complete Day 1 datapath. Label:

```
PC register
instruction memory
field extraction and decoder
immediate generator
two-read/one-write register file
ALU and signed/unsigned compare
data memory
writeback mux
pc+4 and pc+immediate paths
next-PC selection
```

Use two colors: one for data values and one for control selections. Trace `ADD`, `LW`, `SW`, `BLT`, and `JALR` across the drawing.

Observe

Preserve:

- one decoded R instruction;
- one negative B immediate reconstructed from scattered bits;
- the complete loop trace;
- one deliberately unsupported word and the diagnostic;
- one sentence distinguishing model behavior from official exception behavior.

Build

Change the loop's four data words, predict the final sum, and rerun. Then add one `XOR` instruction to the program using the provided encoder. Before running, write its expected instruction fields and architectural update.

Mastery checkpoint

> Starting with one 32-bit instruction word and old architectural state, derive every operand, candidate result, control choice, and state update needed to execute it—then identify which claims came from the ISA and which came from our single-cycle implementation.

Why? notebook

1. Why does an ISA define results but usually not cycle count?
2. Why is a subset not permission to alter an included instruction?
3. Why can `SW` reuse bit positions occupied by `rd` in an R instruction?
4. Why are B- and J-immediate bits rearranged?
5. Why does `JAL` need both `pc+4` and `pc+imm`?
6. Why does `JALR` clear target bit zero?
7. Why do signed and unsigned comparisons need the same register bits?
8. Why can separate instruction/data ports coexist with one software address space?
9. Why is “halt on bad opcode” not an implementation of an illegal-instruction exception?
10. Why does precise exception handling matter to a restartable page fault?

References used selectively

- RISC-V International, *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture*, RV32I base integer chapter, version 20240411: https://docs.riscv.org/reference/isa/unpriv/rv32.html
- RISC-V International, unprivileged ISA index and current ratified specifications: https://docs.riscv.org/reference/isa/unpriv/unpriv-index.html
- RISC-V International, *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture*, for the exception/privilege mechanisms explicitly deferred here: https://docs.riscv.org/reference/isa/priv/priv-index.html
- David Money Harris and Sarah L. Harris, *Digital Design and Computer Architecture: RISC-V Edition*, single-cycle processor chapters, for the pedagogical datapath decomposition.

Next bridge: Day 2 makes three boxes concrete: an ALU that returns values and comparisons, a two-read/one-write register file that enforces `x0`, and a program-counter register whose next value comes from explicit control-flow logic.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Rv32I Subset Trace

Format: Python lab

What you will do: Encode, decode, and trace the documented Day 1 RV32I teaching subset.

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
python3 05-riscv-cpu-microarchitecture/challenges/day-001-rv32i-subset-trace.py
```

- View lab source on GitHub

Day 2 — ALU, Register File, and Program Counter

Target time: approximately 2–3 hours

- ALU results and comparisons: 35–45 minutes
- Register-file read/write contract and `x0`: 40–50 minutes
- Program-counter state and next-PC logic: 35–45 minutes
- Build, simulate, and explain one small RTL lab: 40–50 minutes

> If a register file returns operands continuously but accepts a result only at a clock edge, what exactly happens during one instruction cycle?

Why this day exists

Yesterday derived boxes from the RV32I subset. Today makes three of them precise:

```
register file supplies operands
        ↓
ALU returns a 32-bit result and comparisons
        ↓
writeback becomes architectural state at an edge

PC register supplies current instruction address
        ↓
next-PC logic chooses sequential or redirected address
        ↓
PC becomes new architectural state at an edge
```

The important lesson is not SystemVerilog syntax. It is the boundary between combinational values that respond during a cycle and state that changes at a clock edge.

By the end, you should be able to:

- specify modulo-32-bit ALU behavior;
- distinguish signed from unsigned less-than on identical input bits;
- explain why RV32I uses comparison results and branch conditions rather than a general architectural flags register;
- explain two combinational register-file reads and one synchronous write as a design convention, not an ISA mandate;
- enforce `x0` on reads and writes;
- separate the current PC register from combinational next-PC candidates;
- validate a small ALU/register-file RTL model with Icarus and Verilator;
- state what the lab proves and what remains for a complete CPU.

---

1. The ALU transforms bit patterns

For the Day 1 subset, the ALU needs:

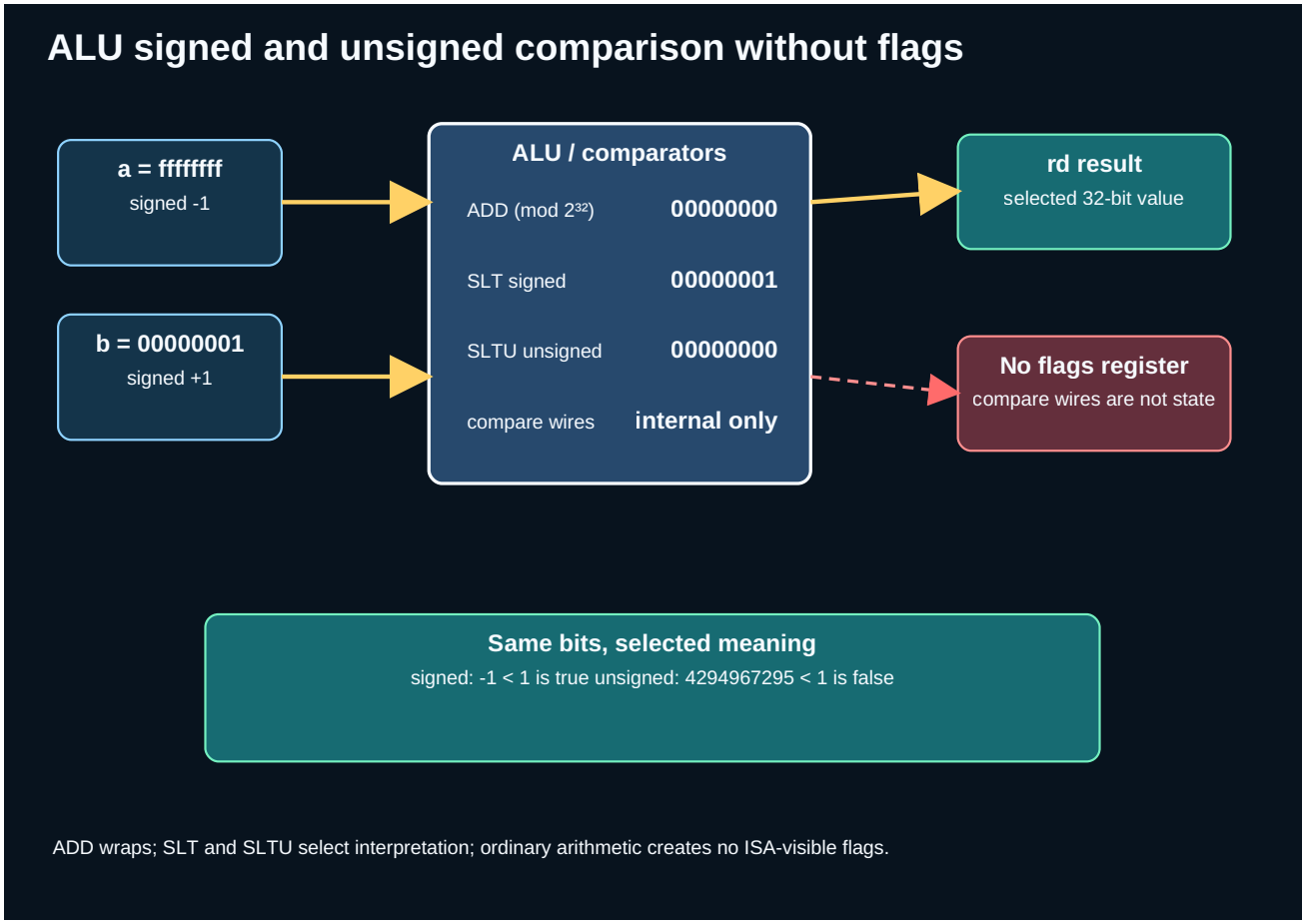
```
ADD   a + b modulo 2^32
SUB   a - b modulo 2^32
AND   bitwise a & b
OR    bitwise a | b
XOR   bitwise a ^ b
SLT   signed(a) < signed(b)  ? 1 : 0
SLTU  unsigned(a) < unsigned(b) ? 1 : 0
```

The ALU receives 32-bit patterns. The selected operation determines their interpretation.

```
a = 0xffffffff
b = 0x00000001

ADD   → 0x00000000
SLT   → 0x00000001    because signed -1 < 1
SLTU  → 0x00000000    because 4294967295 < 1 is false
```

No register carries a permanent signed/unsigned tag. The control signal selecting `SLT` or `SLTU` gives the comparator its meaning.



Identical operand bits produce different comparisons only when control selects their interpretation.

Modulo arithmetic is the RV32I result

`ADD`, `ADDI`, and `SUB` ignore arithmetic overflow in the sense that no integer-overflow exception is raised. The low 32 bits are written:

```
0xffffffff + 1 = 0x00000000   modulo 2^32
0x00000000 - 1 = 0xffffffff   modulo 2^32
```

Signed C overflow is still undefined behavior at the language level. That does not contradict the hardware operation. A compiler may reason that a well-defined C execution never overflows, while the emitted RV32I `ADD` always has a modulo result if executed.

One ALU result, several consumers

The same 32-bit output can serve:

- integer writeback for arithmetic;
- effective address generation for `LW` / `SW`;
- target calculation for `JALR`;
- part of branch-target or `AUIPC` calculation if the implementation shares hardware.

Sharing is an implementation choice. A single-cycle design often adds dedicated PC adders so data arithmetic and next-PC calculation can happen in parallel.

---

2. Compare outputs are not architectural flags

The lab ALU returns:

```
eq            a == b
lt_signed     signed(a) < signed(b)
lt_unsigned   unsigned(a) < unsigned(b)
```

A branch-control block can derive:

```
BEQ   eq
BNE   !eq
BLT   lt_signed
BGE   !lt_signed
BLTU  lt_unsigned
BGEU  !lt_unsigned
```

`SLT` and `SLTU` instead turn a comparison into an integer value:

```
comparison true   result = 0x00000001
comparison false  result = 0x00000000
```

Those are related uses of compare logic, but their destinations differ.

Why not a flags register?

Base RV32I has no x86-style general integer condition-code register updated by ordinary arithmetic. `BEQ` and `BLT` name their source registers directly. That means the architectural dependency is:

```
branch ← rs1, rs2
```

rather than:

```
compare/subtract → architectural flags → later branch
```

Our combinational `eq` and less-than outputs are internal datapath signals. They do not become hidden ISA-visible state merely because the RTL names them.

This distinction matters in a context switch. A kernel must preserve architectural state. It does not save every combinational wire from the branch comparator.

Carry and overflow

An implementation may internally compute carry or signed overflow. Base RV32I `ADD` does not write a carry flag or trap on signed overflow. Multiword arithmetic can derive carry with ordinary instructions, for example by comparing the sum with an operand using `SLTU`.

Do not add architectural flags because an adder can conveniently produce them. Hardware capability does not silently extend the ISA contract.

---

3. The register-file interface

The subset needs at most two source operands and one destination per instruction. A natural register-file interface is:

```
raddr1[4:0] ──► read port 1 ──► rdata1[31:0]
raddr2[4:0] ──► read port 2 ──► rdata2[31:0]

write_enable
waddr[4:0] ──► write port
wdata[31:0] ──► accepted on rising edge
```

Five address bits select one of 32 architectural names.

Our timing convention

The lab chooses:

- **combinational reads** — changing `raddr1` or `raddr2` changes the corresponding read data after combinational delay;
- **synchronous write** — when `write_enable` is true, `wdata` is accepted at the rising edge for `waddr`;
- **two read ports, one write port**;
- **no reset of x1–x31**.

This convention fits the single-cycle path:

```
PC/instruction bits settle
 → rs1/rs2 addresses select register values
 → ALU and memory paths settle
 → result is written at the closing edge
```

It is not mandated by RV32I. An FPGA-oriented design might use block RAM with synchronous reads. A high-performance CPU may use banked, replicated, multiported, renamed, or bypassed structures. Any implementation must preserve architectural behavior.

RTL array, physical cells, and ports

`logic [31:0] registers [31:0]` describes indexed state and the lab's access behavior; it does not declare a literal floorplan of 1,024 independent flip-flops. A synthesis flow for a small target may implement the array with flip-flops and muxes. An FPGA tool may infer distributed RAM or block RAM. An ASIC may use a custom or generated register-file macro whose dense storage cells, wordlines, bitlines, read sensing, and write drivers differ from both.

Ports are physical resources, not just function arguments. Two simultaneous reads and one write may require a true multiported cell, duplicated read structures, banking, time-multiplexing, or bypass logic. Those choices affect area, delay, energy, and collision behavior while preserving the same architectural register semantics. Therefore say “this RTL models a two-read/one-write register file,” not “all register files are flip-flop arrays” or “every RTL port becomes one identical physical port.”

Why there is no bulk reset

RV32I does not promise that x1–x31 become zero at reset. Clearing every entry in a small RTL array can also imply reset hardware that a physical register file would not use.

The testbench therefore writes a register before expecting a known value. In a real boot flow, reset establishes the PC and platform-required control state; firmware initializes the software state it needs.

---

4. `x0` needs two enforcement points

Architecturally:

```
read x0       always returns 0
write x0      has no effect
```

The lab implements both:

```
assign rdata1 = (raddr1 == 5'd0) ? 32'd0 : registers[raddr1];

always_ff @(posedge clk)
    if (write_enable && waddr != 5'd0)
        registers[waddr] <= wdata;
```

Suppressing writes alone is insufficient if the physical entry for index zero is unknown and reads expose it. Forcing reads to zero alone hides an unnecessary write but allows useless state changes and can complicate verification or power reasoning. The contract is clearest when both paths express the invariant.

An implementation may omit physical storage for x0 entirely. The ISA requires behavior, not 32 identical storage rows.

Same-address read and write

Suppose x5 contains `7`, while the rising edge writes `9` to x5.

Under the lab's abstract RTL model:

```
before edge                 rdata for x5 is 7
at edge                     write accepts 9
after nonblocking update    rdata for x5 becomes 9
```

That is useful for this model, but actual memory macros can be read-first, write-first, no-change, or device-specific for a same-address collision. Pipeline bypass logic may be required when a consumer needs a value before the register file naturally presents it.

In the single-cycle CPU, the next instruction is selected after the PC updates and sees the newly written register value during the next cycle. There is no overlap between instructions yet.

---

5. The PC is a register, not the entire control-flow mechanism

The **program counter register** stores the address of the current instruction. Combinational logic computes candidates:

```
sequential    pc + 4
branch        pc + branch_imm
JAL           pc + jal_imm
JALR          (rs1 + i_imm) & ~1
```

A next-PC selector chooses one candidate:

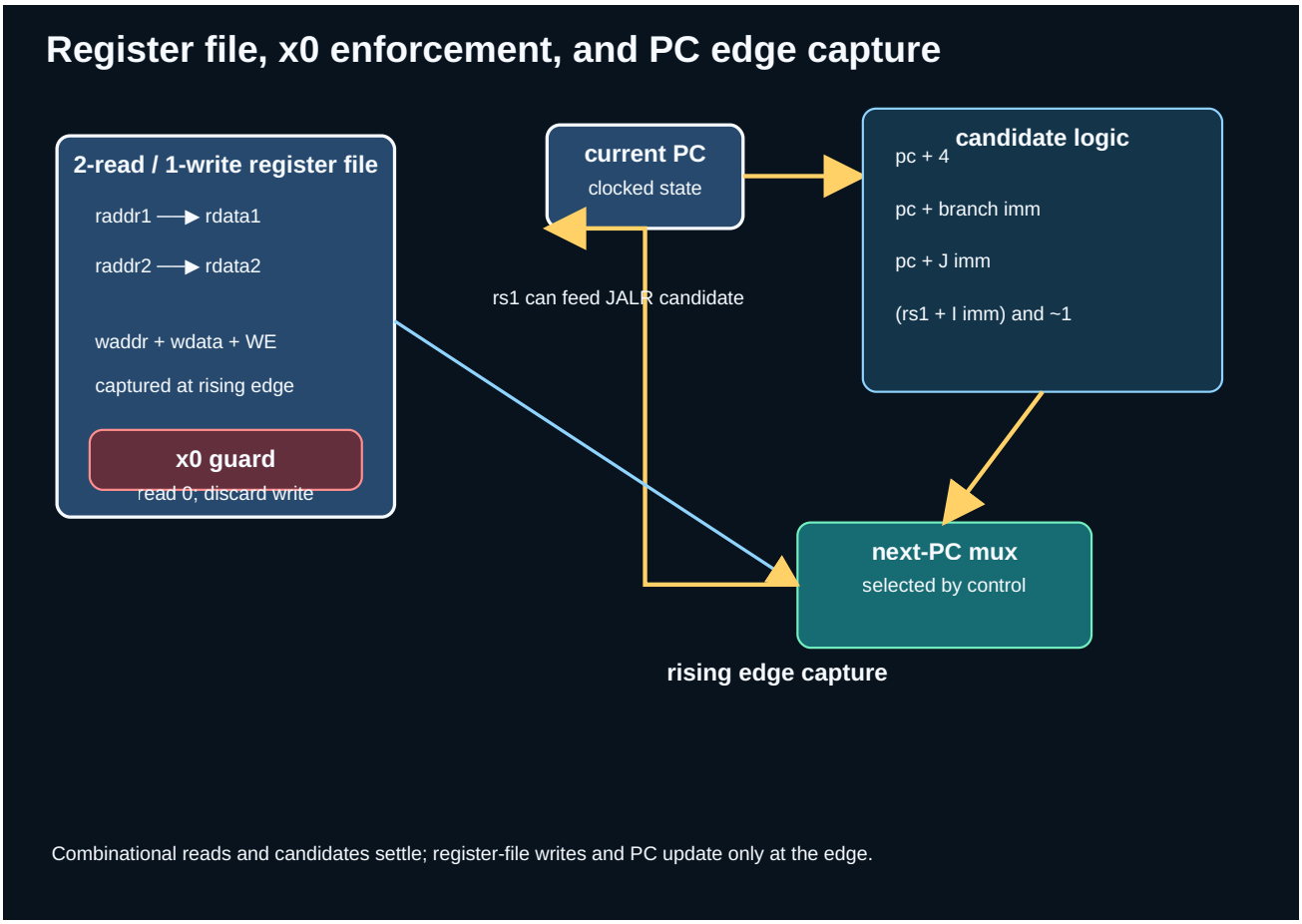
```
next_pc = branch_taken ? branch_target : pc_plus_4
```

with higher-level decode choices for `JAL` and `JALR`.

At the rising edge:

```
pc <= next_pc;
```

Do not say that the PC “increments itself.” An adder creates a candidate `pc+4`; control selects it; the PC register captures it.



Reads and next-PC candidates are combinational; writes and PC replacement occur at the edge.

Current PC and link value

For `JAL` and `JALR`, two values are needed:

```
next PC        control-flow target
rd writeback   old pc + 4
```

The current instruction's PC must remain available while both are computed. If `rd=x0`, the jump still occurs and the link write is discarded.

Reset PC is a platform choice

A complete RTL CPU needs some way to enter a known fetch location, commonly a reset input and parameterized reset vector. The unprivileged ISA does not declare one universal physical reset address for every RISC-V platform.

This Day 2 lab does not instantiate a PC register because its only sequential lab state is the register file. The PC contract is specified and traced here; it will join decode and memory in the complete CPU. Keeping the lab bounded prevents it from becoming a partial, misleading processor.

Architecture comparison — flags, registers, and PC

The three ISAs expose different **architectural** dependencies:

- RV32I has 32 integer registers, with `x0` fixed at zero, and conditional branches compare registers directly. `JAL`/`JALR` can write a link to the selected `rd`; the ABI conventionally uses `ra`.
- AArch64 has 31 general-purpose integer registers plus an encoding that means the zero register or stack pointer by context. `PC` is not a general-purpose register. Some arithmetic forms update `NZCV`, `B.cond` consumes those flags, while instructions such as `CBZ` and `TBZ` test a register without first setting flags; `BL` writes link register `x30`.
- x86-64 has 16 general-purpose registers. `RIP` and `RFLAGS` are architectural state, although `RIP` is not an ordinary integer register. Many arithmetic instructions update flags that later conditional branches consume.

This changes compiler-visible dependence chains and context state, but it does not mandate a physical flags register, register-file port count, or PC pipeline. Renaming, bypassing, and next-PC hardware are **microarchitectural**; ABI register roles and OS save/restore policy are separate contracts. See Architecture Comparison for Systems Programmers.

---

6. Read the small RTL lab

Files:

```
05-riscv-cpu-microarchitecture/challenges/day-002-alu-register-file.sv
05-riscv-cpu-microarchitecture/challenges/day-002-alu-register-file-tb.sv
05-riscv-cpu-microarchitecture/challenges/day-002-run-alu-register-file.sh
```

The design file contains exactly two modules:

```
rv32_alu       combinational value and compare outputs
rv32_regfile   two combinational reads, one rising-edge write
```

It is not an instruction decoder or CPU.

ALU structure

The ALU computes compare outputs continuously and selects one 32-bit result:

```
eq          = (a == b);
lt_signed   = ($signed(a) < $signed(b));
lt_unsigned = (a < b);
```

`$signed` changes how the comparison interprets the vector. It does not add a sign bit or mutate `a`.

The result `case` has a default. Complete assignment avoids accidental storage in combinational logic.

Register-file structure

Read ports are continuous assignments. The write uses `always_ff` and a nonblocking assignment. This directly expresses the chosen timing contract:

```
read selection     combinational
retained update    rising-edge state change
```

There is no testbench delay, print, or assertion inside the design modules. Those simulation-only activities remain in the testbench.

---

7. Predict, run, inspect

From the repository root:

```
bash 05-riscv-cpu-microarchitecture/challenges/day-002-run-alu-register-file.sh
```

The script:

1. checks the design and testbench with Verilator lint;
2. compiles them as SystemVerilog with Icarus Verilog;
3. runs assertions with `vvp`;
4. uses a temporary build directory, leaving source directories clean.

Expected final result:

```
PASS: ALU and register-file checks completed
```

Predict before running

Write answers first:

1. What are ADD, SLT, and SLTU results for `a=0xffffffff`, `b=1`?
2. Does writing `0xdeadbeef` to x0 affect either read port?
3. If x5 holds `7`, what does its combinational read show before and after an edge writing `9`?
4. Does the ALU retain its previous result when `op` changes?
5. Do the compare outputs become architectural state?
6. Does a passing test establish maximum clock frequency?

Controlled break

Make one temporary change at a time:

- remove `waddr != 0` from the write condition;
- remove the read-side x0 selection;
- change signed less-than to an unsigned comparison;
- delete one ALU `case` assignment and the default.

Predict which assertion or lint warning should expose each defect. Restore the source and rerun. A defect not caught by the current tests is evidence that the testbench needs another observation—not that the defect is safe.

---

8. What the tools establish

A clean Verilator run means the selected lint configuration reported no warning or error. A passing Icarus run means the simulated model satisfied the testbench assertions for those cases.

Neither proves:

- every 32-bit operand pair;
- decoder correctness;
- physical register-file implementation;
- setup, hold, clock-to-Q, or maximum frequency;
- same-address behavior of a future SRAM macro;
- exception behavior;
- a complete RV32I processor;
- equivalence to a synthesized netlist.

The testbench checks representative arithmetic boundaries, signed/unsigned comparison, dual reads, synchronous writes, x0, and the before/after-edge behavior. Those are focused claims.

---

9. Systems relevance

These small blocks appear in larger systems questions:

Context switching

The kernel saves architectural registers, not ALU compare wires. `x0` need not be saved as a mutable value because its architectural value is known.

Trap restart

The saved exception PC identifies the instruction that must be diagnosed or restarted according to the trap's semantics. The next-PC decision therefore cannot be treated as incidental bookkeeping.

Emulation and virtualization

An emulator can keep `regs[32]` in host memory and force index zero to zero. Hardware may omit x0 storage. Both implement the same architectural rule by different mechanisms.

Compiler code generation

The absence of flags shapes sequences. Unsigned carry can become an `SLTU` result; branches compare registers directly. The compiler targets ISA behavior, not our particular ALU module.

Debugging

When a trace says the PC jumped incorrectly, separate candidates:

```
bad immediate decode?
bad signed/unsigned compare?
wrong branch control?
wrong target addition?
wrong PC capture?
stale operand from register file?
```

The block contract turns “the CPU is wrong” into testable boundaries.

---

10. Mastery evidence

Explain

Without notes:

- explain why addition wraps without an RV32I overflow trap;
- compare SLT, SLTU, and branch compare outputs;
- explain why internal comparison wires are not architectural flags;
- describe combinational reads and synchronous writes over one cycle;
- enforce x0 independently on read and write paths;
- explain why x1–x31 are not bulk-reset in this lab;
- distinguish current PC, next-PC candidates, selection, and capture.

Draw

Draw:

```
instruction rs1/rs2 fields
      ↓
two-read register file
      ↓ a,b
ALU result + eq/lt_signed/lt_unsigned
      ↓
writeback selection → synchronous write

PC register → pc+4 / pc+imm / (rs1+imm)&~1
      ▲                    ↓
      └──────── next-PC selection
```

Mark combinational paths in one color and edge-triggered state in another.

Observe

Preserve:

- warning-clean Verilator output;
- warning-clean Icarus compile output;
- the testbench PASS line;
- one signed-versus-unsigned comparison prediction;
- one x0 write-discard observation;
- one before-edge/after-edge same-register observation.

Build

Add one test—not one new hardware feature—that checks:

```
0x80000000 < 0x00000000
```

under both signed and unsigned interpretation. Predict all three compare outputs and the `SLT`/`SLTU` result values before running.

Then sketch, without implementing, a PC module with:

```
clk, reset, next_pc, current_pc
```

State whether reset is synchronous or asynchronous and choose a reset vector. Label both as implementation/platform choices.

Mastery checkpoint

> Trace one `SLT`, one `BLTU`, and one `JALR` from register-file read through combinational values to the edge where architectural state changes. Name every value that is merely internal and every value that remains visible to the next instruction.

Why? notebook

1. Why do ADD and SUB not need signed and unsigned variants?
2. Why do SLT and SLTU need distinct interpretations?
3. Why can branch comparison wires exist without an architectural flags register?
4. Why are combinational register-file reads convenient for a single-cycle CPU?
5. Why might an FPGA implementation choose a different read convention?
6. Why must x0 behavior be correct even if physical row zero does not exist?
7. Why can a same-address read/write policy require bypass logic later?
8. Why is resetting every register neither required by RV32I nor always cheap?
9. Why is `pc+4` a combinational candidate rather than an automatic property of the PC?
10. Why can a passing RTL simulation not establish a safe clock period?

References used selectively

- RISC-V International, *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture*, RV32I programmer's model, integer computational instructions, control transfer, and `x0` behavior, version 20240411: https://docs.riscv.org/reference/isa/unpriv/rv32.html
- IEEE Std 1800-2023, *SystemVerilog*, for `always_comb`, `always_ff`, nonblocking assignment, packed vectors, and signed casts: https://standards.ieee.org/ieee/1800/7743/
- Icarus Verilog documentation: https://steveicarus.github.io/iverilog/
- Verilator guide: https://verilator.org/guide/latest/

- David Money Harris and Sarah L. Harris, *Digital Design and Computer Architecture: RISC-V Edition*, processor datapath and register-file sections, for the teaching implementation pattern.

Next bridge: connect these blocks to immediate generation, instruction decode, memories, writeback, and next-PC control, then test complete instruction-level state transitions before introducing pipelining.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Run Alu Register File

Format: shell lab

What you will do: set -euo pipefail

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 05-riscv-cpu-microarchitecture/challenges/day-002-run-alu-register-file.sh
```

- View lab source on GitHub

Day 3 — Decoder, Control, and Single-Cycle Paths

Target time: approximately 2–3 hours

- Fields, immediates, and decode: 40–50 minutes
- Control signals and datapath steering: 45–55 minutes
- Critical-path reasoning: 25–35 minutes
- Decoder laboratory and evidence: 45–55 minutes

> An instruction word does not “tell the ALU what to do.” Which concrete wires must change before that word can update a register, memory, or the PC?

Why this day exists

Systems programmers usually meet an instruction as assembly text or a disassembler line. A processor receives bits. Between those bits and an architectural state change sit field extraction, legality checks, an immediate generator, control logic, muxes, and write enables.

That machinery explains practical facts:

- why a malformed or unsupported encoding must not silently store data;
- why adding an ISA operation changes decode and datapath requirements;
- why a simple-looking load can set the clock period of a single-cycle core;
- why QEMU's decode code resembles, but is not timed like, hardware decode;
- why “the CPU executes `lw`” is shorthand for coordinated state movement.

Today uses a deliberately small RV32I subset:

```
ADD SUB AND OR | ADDI | LW SW | BEQ BNE
```

This is not a Python lesson and not a complete RISC-V implementation.

By the end, you should be able to trace:

```
instruction fields
  → legality and operation decode
  → immediate construction
  → ALU operation and mux selections
  → register/memory enables
  → next-PC selection
```

1. Control and datapath are roles, not separate universes

The **datapath** carries and transforms instruction data:

```
PC → instruction memory → register-file operands → ALU
                                      ↘ data memory ↗
                                      → writeback mux → register file
```

The **control** interprets the instruction and steers that machinery:

```
ALU operation
ALU operand-B select
writeback select
next-PC select
register write enable
memory read/write enables
branch condition kind
illegal-instruction indication
```

This partition is useful for reasoning, but it is not a physical wall. `funct3` wires may run into ALU-control gates; register addresses go directly to the register file; branch comparison can live in the ALU or separate logic. A synthesized implementation may optimize across source-level module boundaries.

Think of control signals as a protocol inside the core. They must be consistent. Selecting memory writeback while not reading memory is suspicious; asserting both memory write and register write for this subset is almost certainly a decoder bug.

---

2. Stable fields begin the decode

For the ordinary 32-bit base formats:

```
opcode  instruction[6:0]
rd      instruction[11:7]
funct3  instruction[14:12]
rs1     instruction[19:15]
rs2     instruction[24:20]
funct7  instruction[31:25]
```

The physical wires can expose all six slices in parallel. Their *meaning* depends on the format. In `SW`, bits 11:7 are immediate bits, not a destination register. In `ADDI`, bits 24:20 belong to the immediate even though an unconditional field extractor can label that slice `rs2`.

That distinction is important when reading traces:

> Extraction says where bits are. Decode says whether those bits have a role.

The seven-bit major opcode first identifies a family. `funct3` and sometimes `funct7` then select an operation within that family. Checking only the major opcode is insufficient. For example, not every `LOAD`-family `funct3` is `LW`, and not every `OP`-family `funct7` is `ADD`.

A two-level view

Many textbook designs describe:

```
main decode(opcode) → broad controls + ALUOp category
ALU decode(ALUOp, funct3, funct7) → exact ALU function
```

That division avoids one giant truth table and reflects reusable operation families. The lab returns the final control bundle directly, but its lookup still performs both conceptual levels. Source organization is not a timing claim; synthesis may flatten or restructure the logic.

Architecture comparison — instruction boundaries and decode work

This lab can index one aligned 32-bit RV32I word before decoding it. With the RISC-V `C` extension, the front end must also distinguish 16-bit and 32-bit instruction lengths. AArch64 A64 keeps a fixed 32-bit width. x86-64 must find boundaries in a variable-length byte stream and decode prefixes, opcode maps, addressing forms, and possible memory operands.

These are **ISA** parsing obligations. They do not prescribe one decoder stage or imply that one family must execute faster. Implementations of all three may use parallel decoders, queues, caches of decoded work, instruction fusion, or internal operations unlike the architectural instruction. The safe-enable rule remains universal: however decode is organized **microarchitecturally**, an illegal or faulting instruction must not acquire unintended architectural effects. See Architecture Comparison for Systems Programmers.

---

3. The immediate generator is format-dependent wiring

RV32I does not place every immediate in one contiguous location. One combinational block can build several candidates and select one according to the decoded format.

```
I: sext(inst[31:20])

S: sext({inst[31:25], inst[11:7]})

B: sext({inst[31], inst[7], inst[30:25], inst[11:8], 1'b0})
```

The B immediate's low bit is zero because branch displacements are encoded in two-byte units. For the base-only core here, valid instruction addresses are four-byte aligned, but the encoding was designed to coexist with 16-bit instructions.

For:

```
bne x3, x0, -16
```

the lab word is `0xfe0198e3`. The immediate generator reconstructs `-16`; the branch target is:

```
current branch PC + (-16)
```

It is not `PC+4-16`, and it is not an absolute address.

Sign extension is wiring in hardware: upper output bits replicate the encoded sign bit. Calling it “free” is too strong—it still contributes routing and fanout—but it need not be a sequential arithmetic procedure.

---

4. A compact control contract

Use these conceptual signals:

```
ALUOp      ADD, SUB, AND, OR
ALUBSel    RS2 or IMM
WBSel      ALU, MEM, or unused
PCSel      PC+4 or branch candidate
RegWrite   permit register-file state update
MemRead    request data-memory read
MemWrite   permit data-memory state update
Branch     BEQ, BNE, or none
Illegal    encoding is not implemented/legal here
```

For the lesson subset:

```
ADD   ALU=ADD  B=RS2 WB=ALU RegWrite=1
ADDI  ALU=ADD  B=IMM WB=ALU RegWrite=1
LW    ALU=ADD  B=IMM WB=MEM RegWrite=1 MemRead=1
SW    ALU=ADD  B=IMM        MemWrite=1
BEQ   ALU=SUB  B=RS2 PC=branch-if-equal
```

`LW` and `SW` use the ALU as an address adder:

```
effective address = register[rs1] + sign-extended immediate
```

The same datapath resource used by `ADD` can therefore serve address generation. Sharing saves hardware, but muxes and longer paths have costs.

Mux selects answer “which candidate?”

A mux does not move data in software sequence. Its candidates exist as combinational signals, and select inputs determine which reaches the output.

For `LW`:

```
ALU A mux:       rs1 value
ALU B mux:       immediate
writeback mux:   data-memory value
next-PC mux:     PC+4
```

For a taken `BNE`:

```
ALU inputs:      rs1 and rs2 for comparison
writeback mux:   irrelevant; RegWrite=0
next-PC mux:     PC + branch immediate
```

An irrelevant select should not be confused with permission. Even if the writeback mux happens to show a value during `SW`, `RegWrite=0` prevents a register update.

Enables are the final safety boundary

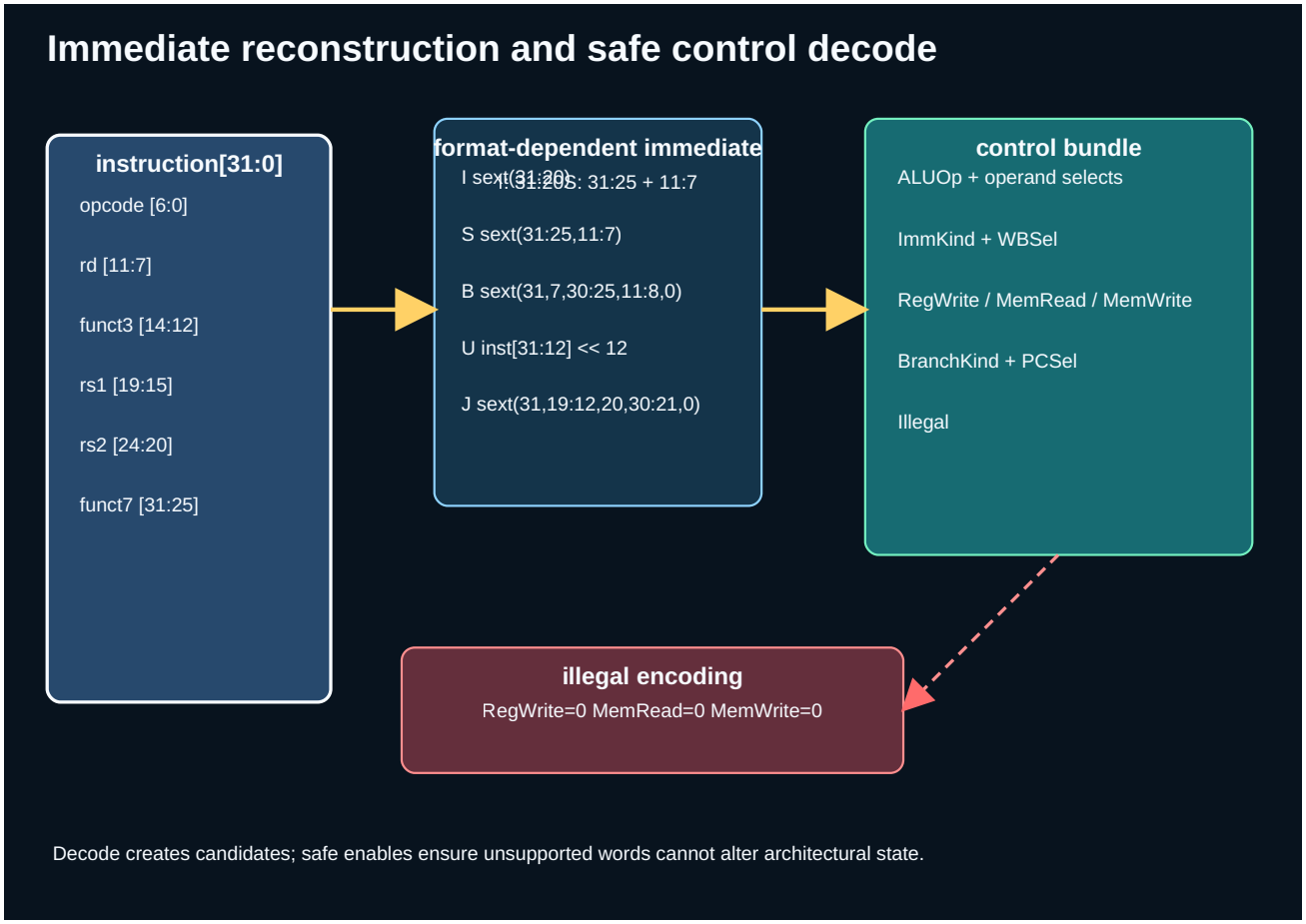
At the active clock edge, architectural state changes only where an enable permits it:

```
if RegWrite: registers[rd] ← writeback value, except x0
if MemWrite: memory[address] ← store data
PC ← selected next PC
```

This is why illegal decode must fail closed. A useful safe default is:

```
Illegal=1, RegWrite=0, MemRead=0, MemWrite=0
```

A real RISC-V core normally raises an illegal-instruction exception rather than merely stopping. The teaching decoder has no privilege or trap machinery, so it reports illegality while proving that write enables remain low.



Legality, immediate construction, selects, and enables must agree before state may change.

---

5. One instruction through the complete path

Trace:

```
lw x4, 0(x2)
```

1. The PC addresses instruction memory.
2. `0x00012203` arrives as the instruction word.
3. `opcode=0000011` identifies `LOAD`; `funct3=010` selects `LW`.
4. `rs1=2`, `rd=4`, and the I immediate is zero.
5. The register file reads `x2`.

6. `ALUBSel=IMM` chooses zero as the ALU's second operand.
7. `ALUOp=ADD` forms the effective address.
8. `MemRead=1` requests the word at that address.
9. `WBSel=MEM` chooses loaded data.
10. `RegWrite=1` allows `x4` to capture it at the edge.
11. The normal next-PC selection chooses `PC+4`.

All combinational steps must settle early enough for the destination register's setup requirement. The numbered explanation is causal, not a list of separate clock cycles.

---

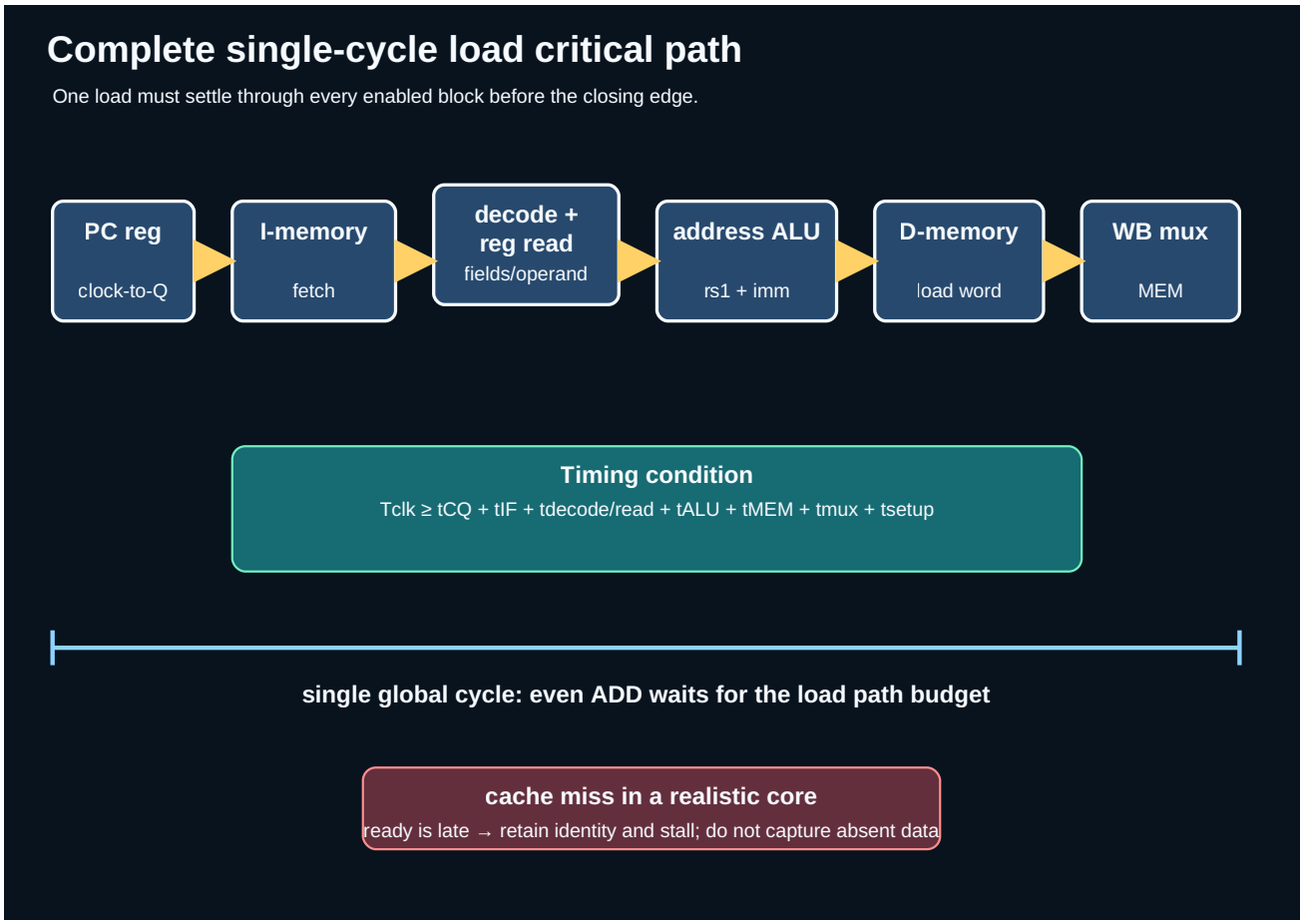
6. The single-cycle critical path

“Single-cycle” means each instruction completes its architectural state transition in one clock period. It does **not** mean the clock period is one nanosecond, every instruction has equal physical delay, or memory responds instantly.

A simplified load path is:

```
PC register clock-to-Q
 → instruction-memory access
 → decode + register-file read + immediate selection
 → ALU address calculation
 → data-memory access
 → writeback mux
 → destination-register setup
```

The minimum clock period must cover the worst enabled register-to-register path, plus clocking uncertainty. The load often dominates a textbook single-cycle design because it crosses two modeled memories and the ALU. An `ADD` does not need the data-memory access, but it still waits for the same global clock edge.



The load crosses both modeled memories, so its full path can set the global period.

A branch has its own competition:

```
PC → instruction decode → register compare → taken decision ┐
PC → immediate generation → target addition                ├→ next-PC mux
PC → PC+4                                                  ┘
```

The select and selected candidate must both arrive in time.

Realistic memories complicate the picture. SRAMs may be synchronous; caches can miss; external memory can take hundreds of cycles. A practical core therefore uses multicycle control, pipelines, caches, handshakes, or some combination. The single-cycle model is valuable because it exposes dependencies, not because it is a competitive implementation template.

Carry this path forward in two steps:

- Day 5 places fetch and data-memory work in pipeline stages, making latency and stage boundaries explicit.
- Day 7 replaces the assumed combinational response with request/response backpressure, cache hit or miss handling, line fill, and a lower-memory return path.

In that realistic timing model, decode still requests `LW`, but `RegWrite` is permission for eventual completion, not permission to capture nonexistent data at the next edge. The core must retain the load's destination and control state while the memory hierarchy responds, suppress writeback on a fault, and commit only when valid data returns. Exact cycle counts are microarchitectural; the load result or exception remains architectural.

7. Laboratory — interrogate the decoder

Use:

```
05-riscv-cpu-microarchitecture/challenges/day-003-control-decoder.py
```

It requires only Python 3's standard library.

Run the checks:

```
python3 05-riscv-cpu-microarchitecture/challenges/day-003-control-decoder.py \
  --selftest
```

Then print the included legal and illegal cases:

```
python3 05-riscv-cpu-microarchitecture/challenges/day-003-control-decoder.py
```

Predict before reading output

For each word, write:

1. opcode family;
2. meaningful register fields;
3. immediate kind and value;
4. ALU operation and operand selections;
5. writeback source;
6. register and memory enables;
7. next-PC source;
8. whether the complete encoding is legal in this subset.

Inspect one branch alone:

```
python3 05-riscv-cpu-microarchitecture/challenges/day-003-control-decoder.py \
  --word 0xfe0198e3
```

Explain why its printed `rd` slice is not a destination and why `RegW=0`.

Illegal-encoding experiment

Compare:

```
python3 05-riscv-cpu-microarchitecture/challenges/day-003-control-decoder.py \
  --word 0x004080b3 \
  --word 0x024080b3
```

The words share the OP major opcode and several fields, but the second has an unsupported `funct7`. Record the write enables for both. This is direct evidence that decoding the major opcode alone is unsafe.

One bounded extension

Add `XOR`:

```
opcode=0110011 funct3=100 funct7=0000000
```

Before editing:

- predict every control output;
- hand-encode or obtain one 32-bit test word;
- add a positive self-test;
- retain a nearby illegal `funct7` test.

Do not add execution behavior today. The task is to extend the control contract without broadening the lab into language practice.

---

8. Evidence and mastery

Preserve:

- the two `PASS` lines from `--selftest`;
- one legal R-type decode;
- one negative immediate reconstruction;
- one load and one store control bundle;
- one branch target immediate;
- two illegal encodings with all architectural write enables low;
- a hand-drawn load critical path.

Explain without notes:

1. Why can fields be physically extracted before their roles are known?
2. Why does `SW` use bits 11:7 without writing `rd`?
3. Why do `LW` and `SW` select an immediate at the ALU B mux?
4. Why are ALU selection and write enable distinct controls?
5. Why must legality include `funct` fields, not only the major opcode?
6. Why can a load determine the clock period for an `ADD`?
7. Which path decides a branch target, and which path decides whether to use it?

Mastery checkpoint

Starting with `0x00012203`, trace every value and control decision from the instruction word to the edge that updates `x4`. Then identify the additional mechanism required if the access is misaligned, unmapped, or denied.

The missing mechanism is an exception path. The tiny decoder does not model one; a production core must suppress ordinary completion, record a cause and faulting PC, and redirect to a trap handler according to its privilege architecture.

Why? notebook

1. Why are register fields kept in stable positions across several formats?
2. Why are immediate bits rearranged instead of shifting an instruction word procedurally?

3. Why does a branch need both a comparison result and a target candidate?
4. Why can a mux output toggle even when its destination write enable is low?
5. Why is “default all writes off” a valuable illegal-decode policy?
6. Why is a Python lookup table evidence about logic behavior but not delay?
7. Why does hardware decode happen concurrently with other combinational work even when a diagram is read left to right?
8. Why does a single-cycle core make fast instructions wait for the longest path?
9. Why does adding an instruction affect verification even if no new datapath unit is needed?
10. Why would QEMU need legality and immediate rules but not this core's clock period?

References used selectively

- RISC-V International, *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture*, “RV32I Base Integer Instruction Set,” for base formats, immediates, integer operations, loads/stores, branches, and illegal instruction behavior: https://docs.riscv.org/reference/isa/unpriv/rv32.html
- RISC-V International, unprivileged ISA specification index and ratified extension status: https://docs.riscv.org/reference/isa/unpriv/unpriv-index.html
- RISC-V International, *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture*, for the real exception/trap mechanisms omitted by the lab: https://docs.riscv.org/reference/isa/priv/priv-index.html
- David Money Harris and Sarah L. Harris, *Digital Design and Computer Architecture: RISC-V Edition*, single-cycle processor and control chapters, for the standard datapath/control decomposition and timing argument.
- QEMU documentation, system emulation and TCG overview, for the later bridge from architectural decode to software translation: https://www.qemu.org/docs/master/devel/tcg.html

Next bridge: connect this decoder to PC state, registers, memory, and an edge-by-edge trace so a hand-encoded loop becomes a complete, observable architectural execution.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Control Decoder

Format: Python lab

What you will do: Control decoder lab for a small RV32I implementation subset.

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
python3 05-riscv-cpu-microarchitecture/challenges/day-003-control-decoder.py
```

- View lab source on GitHub

Day 4 — Integrating and Running a Tiny RV32I Core

Target time: approximately 2–3 hours

- Integrate fetch, decode, execute, memory, and writeback: 40–50 minutes
- Predict and run the hand-encoded program: 50–65 minutes
- Inspect state, branches, and invariants: 35–45 minutes
- Extend, evaluate omissions, and bridge outward: 30–40 minutes

> Can you account for every architectural state change in a complete loop, while keeping the model's convenient assumptions separate from real CPU behavior?

Why this day exists

Yesterday's decoder produced control signals but did not execute a program. Today we connect those signals to a small state machine:

```
architectural state before edge
  → fetch instruction at PC
  → decode fields and controls
  → read registers and form immediate
  → compute ALU, memory, branch, and writeback candidates
  → commit enabled state at one abstract edge
  → architectural state after edge
```

The runnable model makes PC, instruction, control, registers, memory, and next-state candidates visible. It executes a hand-encoded RV32I-subset program that loops over four signed words, sums them, stores the result, and loads it back.

This is a microarchitecture model, not merely an instruction interpreter, because it exposes datapath candidates and an explicit commit boundary. It is still not cycle-accurate for any real processor.

By the end, you should be able to:

- predict the next PC and enabled writes for each supported instruction;
- separate combinational candidates from committed state;
- explain why `x0` remains zero;
- trace little-endian `LW` and `SW`;
- use invariants and final state as evidence;
- name what the model omits;
- relate this loop to QEMU and to a physical pipelined CPU without claiming they work identically.

---

1. The implemented contract

The model supports only:

```
ADD
ADDI
LW
SW
BEQ
BNE
```

It has:

- 32 integer registers, each masked to 32 bits;
- `x0` hardwired to zero;
- a byte-addressed, little-endian data-memory dictionary;
- a list of 32-bit instruction words;
- a byte-addressed PC;
- one architectural instruction committed per model cycle;
- alignment checks for `LW` and `SW`;
- a cycle limit to catch accidental infinite loops.

Execution stops when the PC points just beyond the program list. That is a test-harness convention, not a RISC-V halt instruction.

The distinction matters. Bare RV32I does not promise that “falling off the end” powers down a processor. Real software exits through an environment call, device protocol, firmware convention, debugger, or platform-specific mechanism.

---

2. State versus candidates

Before an edge, the model has architectural state:

```
PC
x0..x31
data-memory bytes
cycle counter (model instrumentation, not an ISA register)
```

Given the current instruction, its combinational phase calculates:

```
decoded fields and control
rs1 and rs2 values
selected ALU operand B
ALU result
optional loaded memory value
optional writeback value
branch decision
next PC
```

No architectural state changes during this calculation. The `clock` method then commits only enabled writes and the selected next PC.

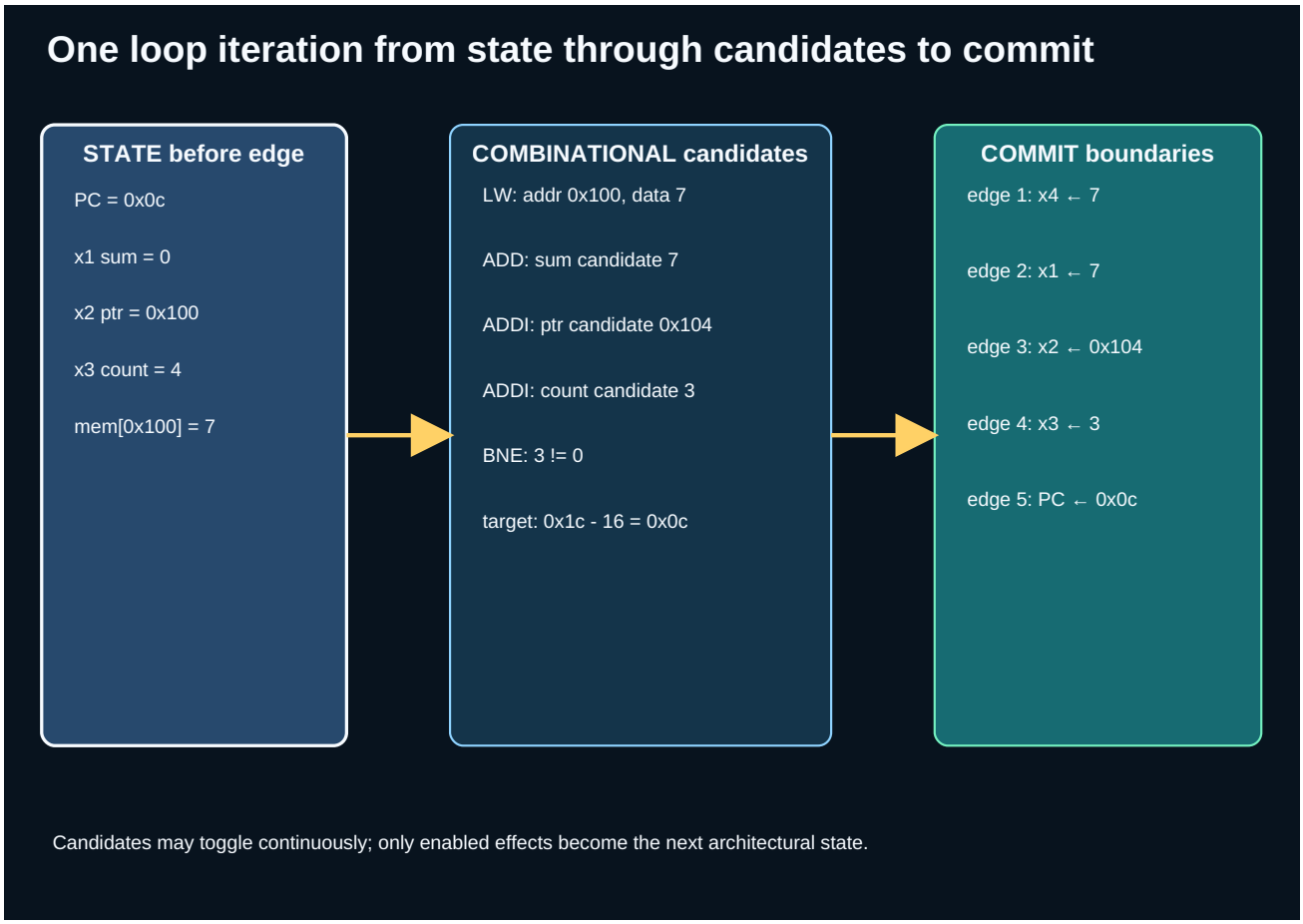
For:

```
addi x3, x3, -1
```

the visible sequence is:

```
before:    x3=4, PC=0x18
candidate: ALU=3, writeback=3, nextPC=0x1c
edge:      x3←3, PC←0x1c
after:     later instructions can observe x3=3
```

The Python statements execute sequentially because software must simulate the model somehow. The represented hardware relationships are combinational settling followed by simultaneous-looking state capture.



Each instruction computes candidates from old state and commits only enabled effects.

`x0` is an invariant

The decoder may identify `rd=0`, and the ALU may calculate a result. The commit path discards the register write when `rd == 0`, then reasserts:

```
registers[0] = 0
```

The invariant is stronger than “this demo does not write x0.” It says every modeled cycle ends with `x0 == 0`.

---

3. The hand-encoded program

The challenge contains these literal words:

```
PC    word        assembly                purpose
00    00000093    addi x1,x0,0            sum = 0
04    10000113    addi x2,x0,256          pointer = 0x100
08    00400193    addi x3,x0,4            count = 4
0c    00012203    lw   x4,0(x2)           load current element
10    004080b3    add  x1,x1,x4           add to sum
14    00410113    addi x2,x2,4            advance pointer
18    fff18193    addi x3,x3,-1           decrement count
1c    fe0198e3    bne  x3,x0,-16          repeat at PC 0x0c
20    00112023    sw   x1,0(x2)           store sum at 0x110
24    00012283    lw   x5,0(x2)           load it back
```

Initial data memory contains:

```
address  signed word
0x100       7
0x104      -2
0x108      13
0x10c       5
```

The sum is 23. The loop advances `x2` four times, so after the final not-taken branch:

```
x2 = 0x110
```

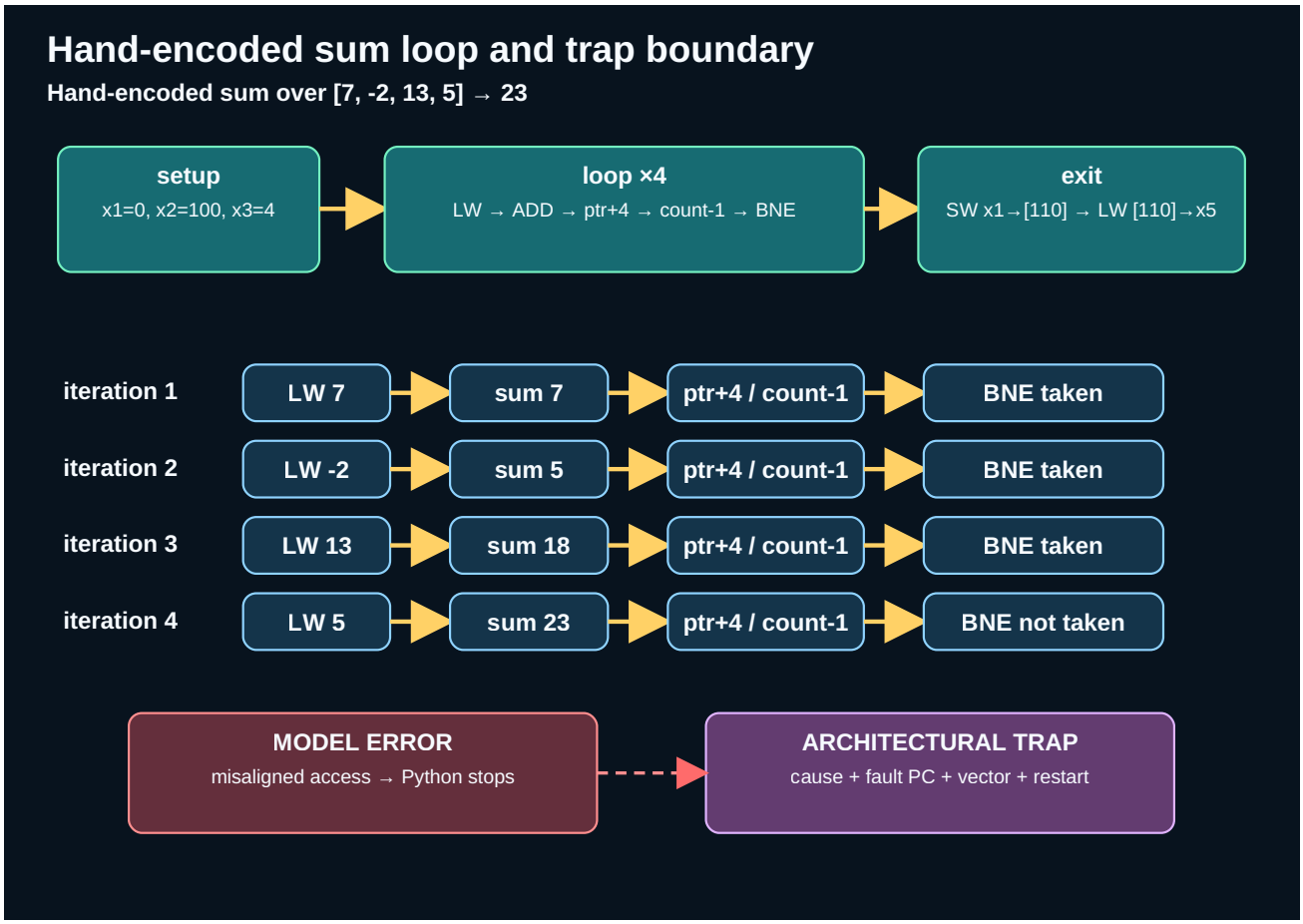
That is where `SW` writes 23 and `LW` reads it into `x5`.

Predict dynamic instruction count

Three setup instructions execute once. Five loop instructions execute four times. Two final memory instructions execute once:

```
3 + (5 × 4) + 2 = 25 model cycles
```

`BNE` is taken three times and not taken once. A wrong prediction here often reveals an off-by-one error about whether decrement happens before the condition is tested.



Four load/add iterations produce 23; a host-model error is not a precise architectural trap.

4. One loop iteration through the datapath

Assume the first iteration begins:

```
PC=0x0c, x1=0, x2=0x100, x3=4
memory word at 0x100 = 7
```

`LW`

```
decode:      I immediate 0, ALU ADD, B=IMM, MemRead, WB=MEM, RegWrite
registers:   A=x2=0x100
ALU:         0x100+0=0x100
memory:      bytes at 0x100..0x103 → 0x00000007
edge:        x4←7, PC←0x10
```

`ADD`

```
decode:      ALU ADD, B=RS2, WB=ALU, RegWrite
registers:   A=x1=0, B=x4=7
candidate:   7
edge:        x1←7, PC←0x14
```

Pointer and count updates

The two `ADDI` instructions produce:

```
x2←0x104
x3←3
```

`BNE`

```
comparison:  x3=3 differs from x0=0
target:      branch PC 0x1c + (-16) = 0x0c
edge:        PC←0x0c
```

The final iteration instead compares zero with zero and chooses `PC+4`, which is `0x20`.

---

5. Loads, stores, and signed interpretation

Memory is byte-addressed and little-endian. The initial `-2` word is stored as the 32-bit pattern:

```
0xfffffffe
```

At increasing addresses:

```
fe ff ff ff
```

`LW` reconstructs the same 32-bit pattern. The register contains bits, not a persistent signed type. The trace displays selected values through a signed helper so humans see `-2`; addition still wraps modulo (2^{32}).

For the second iteration:

```
0x00000007 + 0xfffffffe = 0x00000005  modulo 2^32
```

No overflow flag or trap exists in base `ADD`; the low 32 bits are the architectural result.

`SW` writes the low 32 bits as four bytes. `LW` at the same aligned address reconstructs them, giving a useful end-to-end memory invariant:

```
store_word(address, v); load_word(address) == v & 0xffffffff
```

---

6. Laboratory — predict, run, and explain

Use:

```
05-riscv-cpu-microarchitecture/challenges/day-004-tiny-rv32i-core.py
```

First run the noninteractive checks:

```
python3 05-riscv-cpu-microarchitecture/challenges/day-004-tiny-rv32i-core.py \
  --selftest
```

Then predict:

1. Which cycle first writes `x4`?
2. What are `x1`, `x2`, and `x3` immediately before each `BNE`?
3. Which three branches are taken?
4. At what PC does the final store execute?
5. Which address does it use?
6. What is the final PC?
7. How many model cycles execute?

Run the full trace:

```
python3 05-riscv-cpu-microarchitecture/challenges/day-004-tiny-rv32i-core.py
```

For the first iteration, annotate each line:

```
STATE → CONTROL → COMB candidates → EDGE changes
```

Then use prediction pauses:

```
python3 05-riscv-cpu-microarchitecture/challenges/day-004-tiny-rv32i-core.py \
  --step
```

At every pause, say the exact PC, register, or memory changes before committing. “The ADD executes” is not specific enough.

Fault observation

Make a temporary copy and change the pointer initialization from 256 to 258. Predict the first failing instruction and whether any later state commits. The model reports a misaligned `LW`.

This is a host-language exception used to stop the lab. It is **not** a modeled RISC-V trap: there is no `mepc`, `mcause`, privilege state, trap vector, or restart. Restore the original word after observing the result.

TODO extension

The source leaves one bounded extension:

```
SLT rd,rs1,rs2
opcode=0110011, funct3=010, funct7=0000000
```

Before editing:

- decide whether a new mux is needed;
- specify `ALUOp`, `ALUBSel`, `WBSel`, and enables;
- predict signed results for `-2 < 7` and `7 < -2`;
- hand-encode one test instruction;
- add an independent self-test;
- retain rejection of nearby unsupported encodings.

The key implementation detail is signed comparison of 32-bit patterns. Do not accidentally use Python's nonnegative stored integers as though they already carried RV32 signed meaning.

---

7. What this model omits

The trace is intentionally honest about a small contract. It omits:

- all RV32I instructions except the six named operations;
- instruction bytes and an instruction-memory bus;
- compressed or other extensions;
- privilege modes, CSRs, traps, interrupts, and environment calls;
- virtual memory, protection, an MMU, page tables, and TLBs;
- caches and variable memory latency;
- memory-mapped devices and bus protocols;
- atomic operations and the RISC-V memory model;
- pipelines, hazards, bypassing, stalls, and branch prediction;
- superscalar issue, out-of-order execution, speculation, and retirement;
- realistic SRAM timing, ports, clock-to-Q, setup/hold, and physical delay;
- reset, debug, power, and multicore coherence.

It also treats a dictionary read as combinational data memory. A physical single-cycle implementation cannot assume arbitrary system memory responds within one short clock period.

The trace therefore proves only:

> For this program and modeled initial state, the implemented transition rules produce the asserted final state.

It does not prove RV32I compliance, synthesizability, timing closure, or behavior of an existing CPU.

---

8. Bridge to QEMU

QEMU can run RISC-V software on a non-RISC-V host. At a high level it also:

```
fetch guest instruction bytes
→ decode guest instruction semantics
→ update guest-visible state
```

But QEMU's Tiny Code Generator translates blocks of guest instructions into an intermediate representation and host code, caches translated blocks, and maintains a much broader architectural/platform model. It is not arranging guest mux selects in silicon and does not make one host action equal one guest CPU cycle.

The lab and QEMU share an architectural obligation: given defined starting state and an instruction, produce the specified visible result or exception. They differ in scope, optimization, timing purpose, and implementation.

A useful future experiment is:

```
qemu-system-riscv32 --version
qemu-riscv32 --version
```

if those tools already exist. Do not infer absence of RISC-V support merely because one binary is not installed; system emulation and Linux user-mode emulation are separate builds and use cases.

---

9. Bridge to a real CPU

A simple pipelined implementation could spread today's dependencies across:

```
IF   fetch instruction and form sequential PC
ID   decode, generate immediate, read registers
EX   ALU, address calculation, branch decision
MEM  data-memory access
WB   register writeback
```

That can shorten the combinational work per clock period, but introduces overlap. The loop then creates hazards:

- `ADD` needs the value loaded by the preceding `LW`;
- `BNE` needs the count written by the preceding `ADDI`;
- a taken `BNE` redirects younger fetched instructions;
- `SW` needs the final `x1` sum and `x2` address.

Forwarding can satisfy some dependencies; stalls handle values not ready in time; flushing or redirect logic handles wrong-path fetch. An out-of-order core may execute many internal operations concurrently and speculatively, then retire results in architectural order.

Despite those differences, correct implementations must make committed state look as though this instruction stream obeyed the ISA. That is why the simple trace remains useful to systems programmers: it is a reference-level story of architectural dependence, not a performance model.

---

10. Evidence and mastery

Preserve:

- all three `PASS` lines;
- your predicted and observed dynamic instruction count;
- one taken and the final not-taken branch trace;
- one `LW` showing address and loaded value;
- the `SW` edge showing four-byte state at `0x110`;
- final PC, `x1`, `x3`, `x5`, and memory word;

- one sentence explaining why end-of-list is not an ISA halt;
- one sentence explaining why one model cycle is not one real CPU cycle.

Explain without notes:

1. Which values are state before a cycle?
2. Which values are combinational candidates?
3. Why does the edge update PC even for a store?
4. Why does the loop execute exactly four loads?
5. Why does loading `0xfffffffe` and adding it produce subtraction-like behavior without a signed register type?
6. Why does the final `LW` add confidence beyond inspecting the stored bytes?
7. What would a real misaligned-access exception need to preserve?
8. Which loop dependencies become hazards in a pipeline?

Mastery checkpoint

For the dynamic instruction `BNE` in the third loop iteration, state:

- PC and instruction word;
- decoded register fields and immediate;
- source values;
- control outputs;
- subtraction/comparison result;
- target candidate and sequential candidate;
- selected next PC;
- all architectural state changes at the edge.

Then explain how QEMU and a five-stage core could both preserve that architectural result while doing very different work internally.

Why? notebook

1. Why does a runnable state-transition model reveal more than a decoder table?
2. Why are combinational values printed before edge changes?
3. Why must `x0` be enforced at commit even if ordinary programs rarely target it?
4. Why is the branch target based on the branch instruction's PC?
5. Why does the pointer end at `0x110` rather than `0x10c`?
6. Why is `-2` stored as bytes `fe ff ff ff` on this little-endian model?
7. Why does a cycle limit provide useful validation evidence?
8. Why is a Python alignment error not a RISC-V architectural trap?
9. Why can a pipelined CPU complete the same program with overlapping work?
10. Why can QEMU preserve ISA behavior without reproducing hardware control wires or cycle timing?
11. Why does final state alone not prove every intermediate control decision was correct?
12. Why does passing this program not establish ISA compliance?

References used selectively

- RISC-V International, *RV32I Base Integer Instruction Set*, for integer register state, instruction encodings, loads/stores, branches, and architectural behavior: https://docs.riscv.org/reference/isa/unpriv/rv32.html
- RISC-V International, *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture*, for exceptions, CSRs, and trap behavior omitted from the model: https://docs.riscv.org/reference/isa/priv/priv-index.html
- RISC-V International, *RVWMO Memory Consistency Model*, for the multicore ordering contract not represented by a single-thread dictionary: https://docs.riscv.org/reference/isa/unpriv/rvwmo.html
- QEMU documentation, “Translator Internals” and Tiny Code Generator documentation, for block translation and the distinction from a cycle-accurate hardware model: https://www.qemu.org/docs/master/devel/tcg.html
- QEMU documentation, RISC-V system emulator target: https://www.qemu.org/docs/master/system/target-riscv.html
- David Money Harris and Sarah L. Harris, *Digital Design and Computer Architecture: RISC-V Edition*, processor and pipelining chapters, for the single-cycle-to-pipeline bridge.

Next bridge: pipeline this architectural work, then confront data, control, and structural hazards without changing the software-visible result.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Tiny Rv32I Core

Format: Python lab

What you will do: Runnable, cycle/state-visible single-cycle RV32I-subset core.

This is starter code. Open the file, read its instructions, complete the `TODO` sections, and then run the self-checks.

- Open the lab file

Copy and run:

```
python3 05-riscv-cpu-microarchitecture/challenges/day-004-tiny-rv32i-core.py
```

- View lab source on GitHub

Day 5 — Pipelining and Throughput

Target time: approximately 2–3 hours

- Derive a pipeline from a critical path: 45–55 minutes
- Trace the in-order five-stage model: 45–55 minutes
- Run the timing and visual-trace lab: 35–45 minutes
- Architectural-state, systems, and mastery work: 30–40 minutes

> If one instruction still needs fetch, decode, execute, memory, and write-back, how can a processor finish nearly one instruction every cycle?

Why this day exists

A single-cycle processor places all work for one instruction between two architectural state updates. Its clock must wait for the slowest instruction's longest combinational path. Much of the datapath is idle during parts of that long interval.

Pipelining changes the schedule, not the ISA:

```
long register-to-register path
    → split the combinational work
    → insert clocked boundaries
    → overlap different instructions
    → improve potential throughput
```

It does not make one instruction's work disappear. It adds state, clock overhead, startup time, and control obligations. Today derives those costs before introducing hazards on Day 6.

By the end, you should be able to:

- derive a clock period from stage delays and register overhead;
- distinguish instruction latency from throughput;
- draw the IF/ID, ID/EX, EX/MEM, and MEM/WB boundaries;
- calculate pipeline fill and drain time;
- trace independent instructions through an in-order five-stage pipeline;
- explain why pipeline depth has diminishing returns;
- distinguish architectural state from speculative or intermediate state;
- connect retirement to precise state, performance counters, kernels, and virtualization.

---

1. Begin with the unsplit critical path

Consider a teaching RV64 processor that completes an instruction in one long cycle:

```
PC register
  → instruction memory
  → decode and register-file read
  → ALU / branch compare
  → data memory for a load
  → write-back selection
  → architectural register file
```

Not every instruction uses every block. An `add` does not need data memory, and a store does not write an integer register. The shared clock nevertheless must accommodate the longest valid path used by any supported operation.

Use this illustrative delay budget:

```
instruction fetch                   180 ps
decode and register read            220 ps
execute / address / branch work     250 ps
data-memory access                  210 ps
write-back selection                140 ps
total combinational work           1000 ps
launch/capture and clock overhead    60 ps
```

For the simplified single-cycle machine:

```
Tsingle ≥ 1000 ps + 60 ps = 1060 ps
```

Its ideal instruction rate is one instruction per 1060 ps. The exact numbers are a teaching model, not measurements of a current core.

Why not simply shorten the clock?

The capture edge would arrive before the worst required result became stable. That is a timing failure, not a harmless request for more performance. Software cannot repair a late internal signal.

---

2. Split the path and pay for every boundary

Insert registers around five groups of work:

```
        IF          ID          EX          MEM         WB
PC → fetch → |R| → decode → |R| → ALU → |R| → memory → |R| → select/write
              IF/ID       ID/EX       EX/MEM       MEM/WB
```

Each stage now has one cycle to perform its local combinational work. The period is bounded by the slowest stage, not by the sum:

```
Tpipeline ≥ max(tIF, tID, tEX, tMEM, tWB) + treg
```

For the numbers above:

```
Tpipeline ≥ max(180, 220, 250, 210, 140) + 60
          = 310 ps
```

The register overhead includes a simplified allowance for clock-to-Q, setup, and clock uncertainty. Real timing closure checks individual paths and clock relationships rather than adding one universal constant.

The split is never perfectly free or balanced

The ideal five-way split of 1000 ps would put 200 ps in every stage. Our EX stage needs 250 ps, so 50 ps of capacity in another stage cannot automatically help it. Logic dependencies, memory interfaces, wiring, fan-out, and physical placement constrain where boundaries can go.

Every added stage also costs:

- storage bits and clock-tree load;
- clock-to-Q and setup time;
- control and valid/kill metadata;
- energy on every active cycle;
- additional instruction latency;
- more in-flight work to recover after a redirect or exception.

If useful logic per stage becomes comparable to register overhead, further splitting offers little frequency gain.

---

3. Latency and throughput answer different questions

Latency asks how long one instruction takes from entry to completion. **Throughput** asks how frequently completed instructions can emerge.

For an ideal five-stage pipeline with a 310 ps clock:

```
latency in cycles = 5 cycles
latency in time   = 5 × 310 ps = 1550 ps

steady-state throughput = 1 instruction / 310 ps
```

The pipelined instruction has *higher* time latency than the 1060 ps single-cycle instruction in this example. Throughput improves because several instructions occupy different stages concurrently.

This is the same distinction seen in systems:

- a storage device may have high single-request latency but high queue throughput;
- a network path may have long round-trip time but sustain many bytes per second;
- a CPU may overlap many instructions while one dependent chain remains latency-bound.

A dependency chain sees latency

If each operation needs the previous operation's result, overlap is limited by when that result becomes available. Independent work can exploit throughput; a serial recurrence cannot simply claim the advertised issue width or clock rate.

---

4. Fill, steady state, and drain

Trace six independent instructions:

```
        C1  C2  C3  C4  C5  C6  C7  C8  C9  C10
I1      IF  ID  EX  MEM WB
I2          IF  ID  EX  MEM WB
I3              IF  ID  EX  MEM WB
I4                  IF  ID  EX  MEM WB
I5                      IF  ID  EX  MEM WB
I6                          IF  ID  EX  MEM WB
```

The first result appears only after the pipeline fills. The last instruction must pass through the remaining stages while the pipeline drains.

For `k` stages and `N ≥ 1` independent instructions:

```
ideal cycles = k + N - 1
```

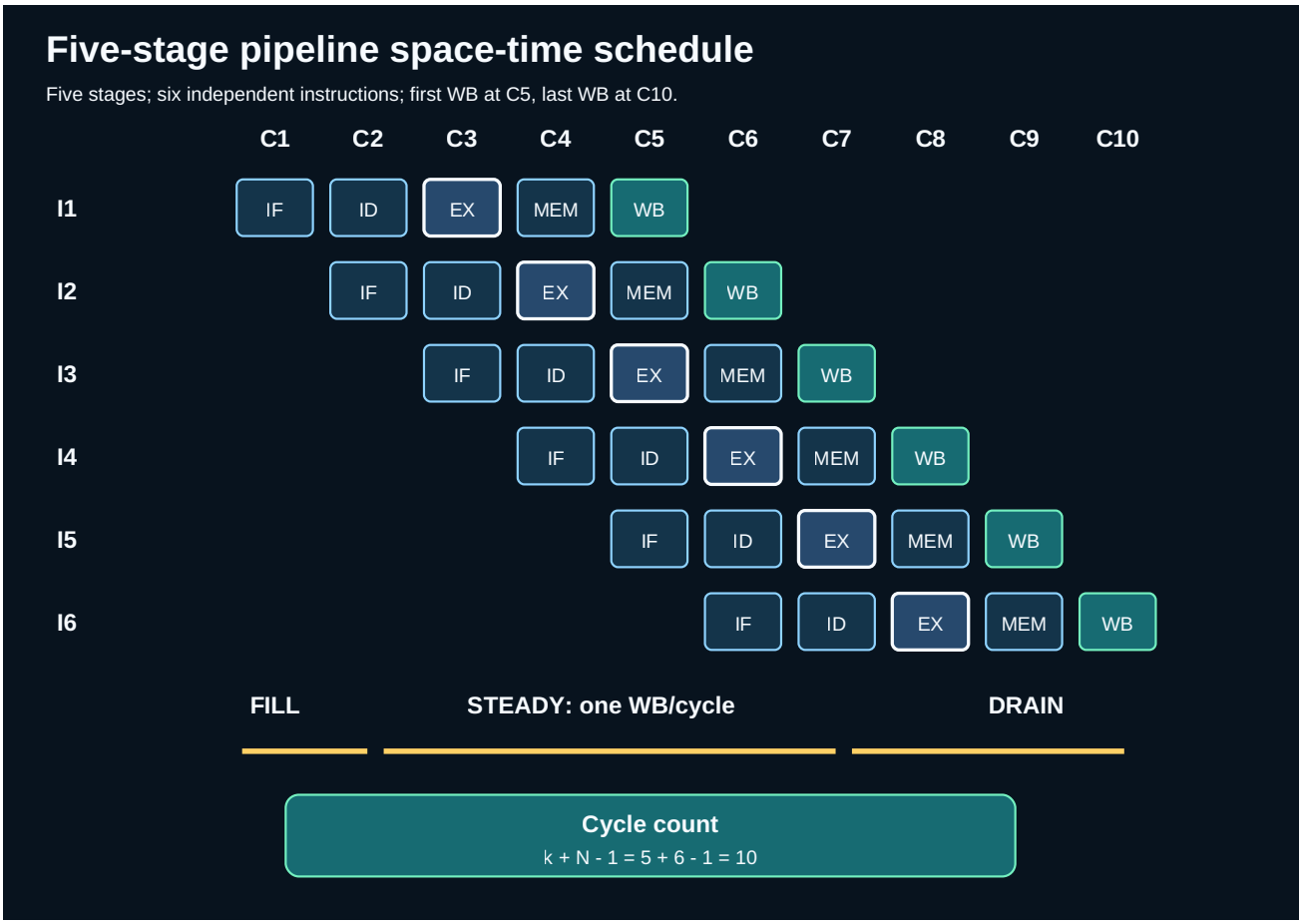
For six instructions and five stages:

```
cycles = 5 + 6 - 1 = 10
```

Average CPI for this finite group is:

```
CPI = cycles / instructions = 10 / 6 ≈ 1.67
```

As `N` becomes large, ideal CPI approaches 1. It does not equal 1 for a short cold sequence, and Day 6 will add bubbles that keep real CPI above the ideal.



Six instructions need ten cycles: five stages plus six instructions minus one.

Batch speedup

For 100 independent instructions:

```
single-cycle time = 100 × 1060 ps = 106000 ps
pipelined time    = (5 + 100 - 1) × 310 ps
                  = 32240 ps
speedup           ≈ 3.29×
```

The speedup is not 5× because:

1. the stages are uneven;
2. register overhead is paid in every pipeline cycle;
3. the finite sequence fills and drains.

---

5. What each stage means

The names describe useful responsibilities, not a universal physical law.

IF — instruction fetch

Conceptually:

```
instruction bits ← memory/cache at PC
candidate next PC ← PC + instruction length
```

The teaching model assumes fixed-width instructions. RISC-V implementations supporting compressed instructions must account for variable 16-bit and 32-bit instruction lengths. Current high-performance cores may fetch blocks, predict multiple control transfers, and queue decoded operations. The five-stage name remains a reasoning scaffold.

ID — decode and register read

ID identifies operation fields, reads source registers, forms immediate values, and carries control metadata forward:

```
instruction bits
    → operation, rs1, rs2, rd, immediate, control
```

EX — execute and address generation

EX performs integer ALU work, comparisons, effective-address generation, and, in this model, branch resolution.

MEM — data-memory access

Loads read and stores present an address/data operation. ALU instructions still occupy the stage even if they merely carry a result through it.

WB — write-back / teaching retirement point

An integer result may update `rd`. Stores and branches do not write an integer register but still complete in program order. We use WB as a simple retirement boundary so the architectural story remains explicit.

Real machines do not have to use these exact boundaries. Some combine stages, split fetch or memory access, use multiple execution pipelines, or retire via a reorder buffer. ISA conformance constrains visible behavior, not these five labels.

Architecture comparison — pipeline shape is implementation freedom

RISC-V, AArch64, and x86-64 define architectural instruction effects and exception/ordering contracts; none defines an IF/ID/EX/MEM/WB pipeline. A small implementation of any of them may be in-order, while a high-performance implementation may be deep, wide, speculative, and out of order.

Their **ISA** front ends begin with different work: base RISC-V and A64 have regular 32-bit encodings, optional RISC-V compressed instructions add 16-bit lengths, and x86-64 has variable-length instructions that are commonly translated to internal micro-operations. After decode, all three permit substantial **microarchitectural** freedom: internal operations, queues, physical-register renaming, execution ports, and retirement structures are not software-visible. The obligation is precise architectural behavior, not similar pipeline diagrams. See Architecture Comparison for Systems Programmers.

---

6. Pipeline registers carry identity as well as data

A common incomplete drawing puts only an ALU value in the stage registers. The machine must also preserve enough metadata to know whose value it is.

```
IF/ID:
  PC, instruction bits, valid/prediction metadata

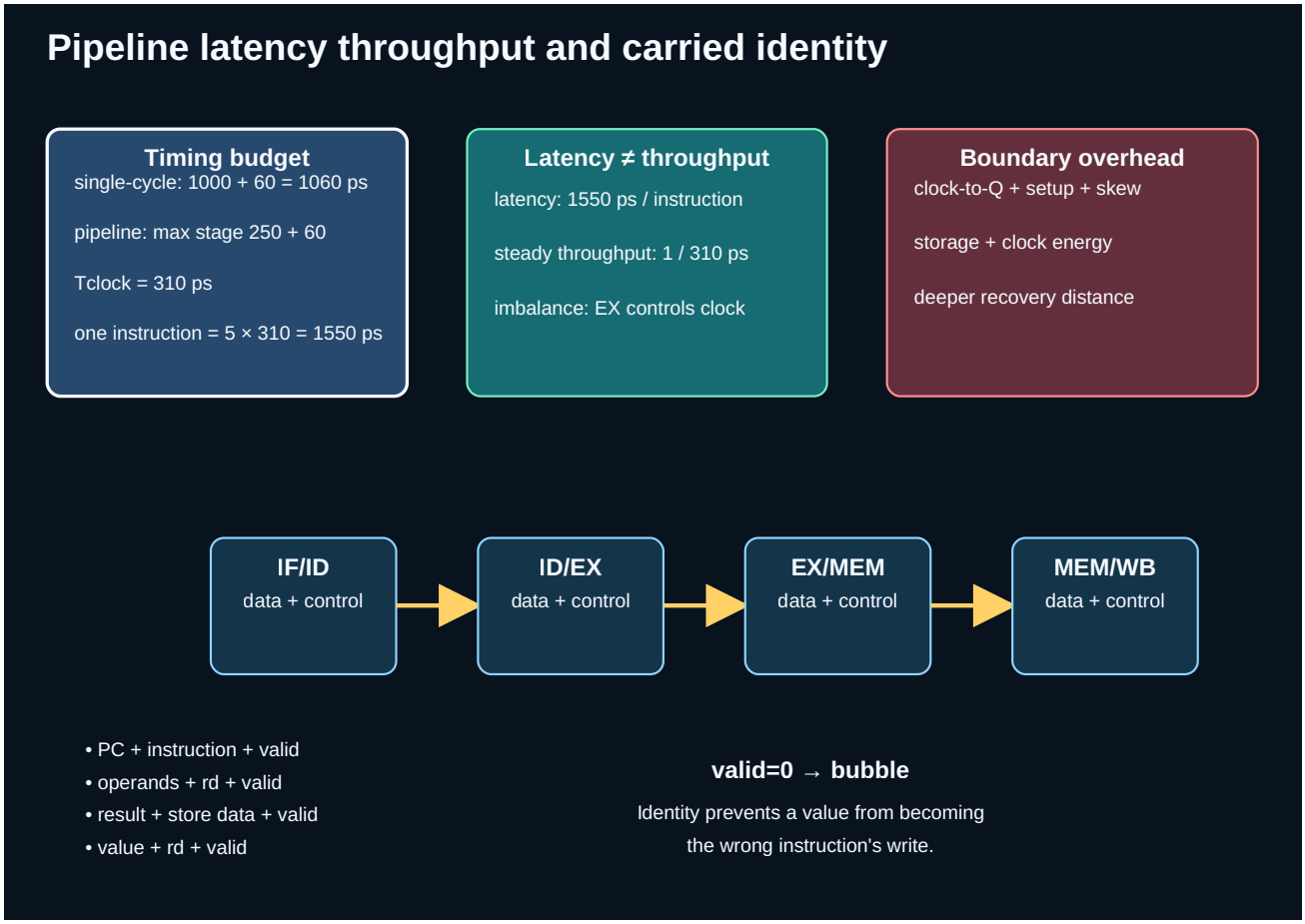
ID/EX:
  operand values, immediate, rd, operation, memory/write controls, PC, valid

EX/MEM:
  ALU/address result, store data, rd, controls, exception metadata, valid

MEM/WB:
  load or ALU result, rd, write control, exception metadata, valid
```

A **valid bit** distinguishes a real instruction from a bubble. A **kill** or flush action invalidates younger wrong-path work. Without identity and validity, a number arriving at WB would not say whether it should update `x9`, write memory, raise an exception, or be ignored.

The values in these registers are microarchitectural state. Software observes their eventual architectural effects, not the registers themselves.



Shorter stage work improves throughput only after paying boundary overhead and preserving identity.

7. Architectural state and the first retirement model

Architectural state is the state promised by the ISA and privilege architecture:

- integer and other architected registers;
- the architecturally defined PC/sequencing behavior;
- memory effects under the architecture's ordering rules;
- privilege, interrupt, exception, and control state where applicable.

Pipeline registers are implementation state. Several instructions may be in flight, but software needs behavior equivalent to the specified program order for this simple in-order machine.

We will use **retirement** to mean:

> the point at which an instruction's result is accepted as part of the non-speculative architectural history.

In the simplified pipeline, completion and retirement are nearly coincident and occur in order near WB. In an out-of-order core, execution may finish in a different order while retirement restores in-order architectural commitment.

Why precise state matters

Suppose an instruction faults. A precise exception boundary presents state as if:

```
all older instructions completed
the faulting instruction did not complete
no younger instruction completed
```

That contract lets an OS identify the faulting instruction, deliver a signal, page memory in, emulate an operation, or resume a guest. Day 6 connects flushing and exception metadata to this boundary.

“WB writes a register” is therefore only the beginning. Stores, exceptions, interrupts, CSRs, and speculative work require a disciplined commitment policy.

---

8. Why this model still matters on current machines

Modern application processors are usually deeper, wider, speculative, and out of order. They may translate ISA instructions into internal operations, use separate pipelines by operation class, and retire several instructions per cycle. The five-stage model remains useful because it isolates durable questions:

- Where is a value produced?
- When may a dependent consumer use it?
- Which work is older or younger?
- Which state is speculative?
- At what boundary does an effect become architectural?
- Which resource limits throughput?

Those questions recur in Intel and AMD optimization manuals, Arm pipeline and PMU documentation, and RISC-V implementations even though the stage names and latencies differ.

Do not infer a current CPU's exact pipeline depth from this lesson. Vendor optimization guides and measured performance-counter evidence are the appropriate sources for a particular core.

---

9. Kernel, `perf`, and virtualization relevance

Performance accounting

Linux `perf stat` can report cycles and instructions when the platform PMU supports them:

```
perf stat -e cycles,instructions -- command
```

A commonly computed ratio is:

```
observed CPI = counted cycles / counted retired instructions
observed IPC = counted retired instructions / counted cycles
```

Interpret those counts cautiously:

- event names and semantics are architecture- and PMU-specific;
- multiplexing can scale counts when events exceed available counters;
- frequency changes make “cycles” different from wall-clock time;
- kernel, hypervisor, guest, and user filtering changes what is counted;
- speculation may consume resources without increasing retired instructions;
- a process can migrate between cores unless constrained.

The pipeline equations explain the baseline. PMU documentation defines the actual evidence.

Precise events

Profilers want to attribute an event to the responsible instruction. A sampled interrupt may arrive after the event because detection, buffering, and interrupt delivery take time. Architecture features such as Intel PEBS or Arm statistical profiling provide more precise mechanisms for selected events. “Precise” is a documented property, not something inferred from one PC sample.

Virtual CPUs

A hypervisor presents architectural CPU state to a guest while scheduling virtual CPUs on physical cores. Pipeline state is generally not part of the guest architectural interface. On a VM exit, hardware and the hypervisor preserve the defined guest state, not every transient stage latch.

This separation is what makes different physical microarchitectures capable of running the same guest ISA.

---

10. Lab — derive, trace, and challenge the ideal

File:

```
05-riscv-cpu-microarchitecture/challenges/day-005-pipelining-and-throughput.py
```

It uses only Python's standard library.

Predict before running

For stage delays `180, 220, 250, 210, 140 ps` and overhead `60 ps`, write:

1. the pipeline clock period;
2. latency in cycles and picoseconds;
3. cycles for 12 independent instructions;
4. whether one-instruction latency improves;
5. why batch speedup is less than five.

Then run:

```
python3 05-riscv-cpu-microarchitecture/challenges/day-005-pipelining-and-throughput.py predict
python3 05-riscv-cpu-microarchitecture/challenges/day-005-pipelining-and-throughput.py predict --
answers
python3 05-riscv-cpu-microarchitecture/challenges/day-005-pipelining-and-throughput.py trace --
instructions 8
python3 05-riscv-cpu-microarchitecture/challenges/day-005-pipelining-and-throughput.py compare
python3 05-riscv-cpu-microarchitecture/challenges/day-005-pipelining-and-throughput.py selftest
```

Observe

Preserve:

- the first cycle in which `I1` reaches WB;
- the last cycle in which `I8` reaches WB;
- the calculated period and one-instruction latency;
- the 100-instruction batch speedup;
- one sentence explaining why the trace is idealized.

Extend without changing the file

On paper, split EX from 250 ps into two 145 ps logic stages. Recalculate:

```
new stage count
new clock period
new one-instruction latency
cycles and time for 1, 8, and 100 instructions
```

The result should make the trade explicit: a higher possible throughput can coexist with greater latency and larger fill/drain cost.

---

11. Evidence: Explain, Draw, Observe, Connect

Explain

Without notes:

1. derive the pipelined clock equation from register-to-register timing;
2. distinguish latency, throughput, CPI, and clock period;
3. explain why the slowest stage controls the clock;
4. derive `k + N - 1`;
5. explain why pipeline registers need metadata and valid bits;
6. distinguish execution completion from architectural retirement.

Draw

Draw the five stages and all four interstage registers. Put one instruction in each stage. Annotate:

- the oldest and youngest instruction;
- one architectural register write;
- one microarchitectural result;
- the retirement point;
- where a bubble would be represented.

Observe

Run the lab and reconcile every number with an equation. A screenshot of the trace without the derivation is not sufficient evidence.

Connect

Choose one:

- a `perf stat` CPI observation;
- a page-fault restart;
- a VM exit preserving guest state;
- a latency-bound pointer chase;
- a throughput-oriented independent loop.

Explain which part of today's model helps and which part the five-stage model omits.

---

12. Mastery checkpoint

Answer:

> Why can pipelining improve instruction throughput while making one instruction take longer, and what state must be carried to preserve the ISA?

A complete answer includes:

- critical-path splitting;
- the maximum stage delay plus boundary overhead;
- overlap among different instructions;
- fill and drain;
- latency versus throughput;
- stage imbalance and register cost;
- interstage data, identity, control, and validity;
- in-order retirement and precise architectural state.

Why? notebook

1. Why does unused time in one stage not automatically repair another stage's critical path?
2. Why does adding a register both help timing and add delay?
3. Why can the first result arrive later while the hundredth arrives sooner?
4. Why is ideal CPI 1 an asymptote for a finite pipeline run?
5. Why must `rd` travel with an ALU value?
6. Why can a bubble occupy a cycle without representing an instruction?
7. Why is a pipeline register not normally architectural state?
8. Why does an OS need a precise exception boundary?
9. Why can retired-instruction counts differ from fetched instruction counts?
10. Why is the five-stage model useful but not a literal diagram of a current high-performance CPU?

References used selectively

- RISC-V International, *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture*, current ratified specifications and instruction semantics: https://docs.riscv.org/
- RISC-V International, *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture*, trap and architectural-state contracts: https://docs.riscv.org/
- David A. Patterson and John L. Hennessy, *Computer Organization and Design: RISC-V Edition*, processor chapter, for the pedagogical five-stage datapath, pipeline timing, and hazards.
- Linux kernel documentation, “Perf events and tool security,” for PMU access and event-accounting context: https://docs.kernel.org/admin-guide/perf-security.html
- Linux `perf-stat(1)` manual, for current command options, event grouping, filtering, and metric reporting: https://man7.org/linux/man-pages/man1/perf-stat.1.html
- Intel, *64 and IA-32 Architectures Optimization Reference Manual*, current core-specific latency, throughput, front-end, retirement, and PMU guidance: https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html

- AMD, *Software Optimization Guides*, current processor-family optimization and performance-monitoring material: https://www.amd.com/en/search/documentation/hub.html
- Arm, *Learn the architecture: Understanding the Armv8.x and Armv9.x extensions*, and PMU documentation, for current architectural and profiling context: https://developer.arm.com/Architectures

Next bridge: overlap creates conflicts. Day 6 classifies structural, data, and control hazards, then uses forwarding, stalls, prediction, and flushing to preserve the in-order architectural result.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Pipelining And Throughput

Format: Python lab

What you will do: Explore five-stage pipeline timing, fill/drain, latency, and throughput.

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
python3 05-riscv-cpu-microarchitecture/challenges/day-005-pipelining-and-throughput.py
```

- View lab source on GitHub

Day 6 — Hazards, Forwarding, Stalls, and Branches

Target time: approximately 2–3 hours

- Classify structural, data, and control hazards: 40–50 minutes
- Derive forwarding, stalls, and branch recovery: 50–60 minutes
- Run and explain the visual simulator: 40–50 minutes
- CPI, precise state, and systems evidence: 30–40 minutes

> Overlap improves throughput only if the machine prevents one instruction's timing from changing another instruction's architectural meaning.

Why this day exists

Day 5 allowed independent instructions and unlimited access to every needed resource. Real programs immediately violate those assumptions:

```
one instruction produces a value another needs
two stages request one resource
a branch changes which instruction should have been fetched
an exception says younger work must never become visible
```

These are not rare edge cases. Dependencies carry program meaning. Loads, branches, cache behavior, and resource conflicts shape delivered performance.

Today keeps the in-order five-stage model and adds only enough control to make overlap correct:

- forwarding when a result exists but has not reached the register file;
- a one-cycle stall for the classic load-use case;
- flushing after a wrong branch prediction;
- valid/kill state so wrong-path instructions cannot commit;
- in-order retirement reasoning for exceptions and precise state.

There is no HDL today. The executable evidence is a dependency-aware Python pipeline simulator.

---

1. A hazard is a timing threat, not necessarily a bug

A **hazard** is a condition that can prevent the next overlapped instruction from proceeding in the intended cycle while preserving correct behavior.

Three broad classes are useful:

```
structural hazard   required hardware resource is unavailable
data hazard         required value is unavailable at the needed time
control hazard      correct next instruction address is not yet known
```

The presence of a dependence does not prove the pipeline must stall. Hardware may provide enough ports, forward a value, predict a branch, or otherwise satisfy the need. A hazard exists when the chosen microarchitecture cannot honor the desired schedule directly.

---

2. Structural hazards come from finite resources

Suppose IF and MEM share one single-ported memory:

```
IF:   read next instruction
MEM:  read load data
```

If a load reaches MEM while another instruction needs IF, both request the same port in the same cycle. One must wait.

Possible designs include:

- separate instruction and data memories or L1 caches;
- a multi-ported memory;
- arbitration plus an IF stall;
- buffering that decouples fetch from later consumption.

Every response has costs in area, power, timing, or complexity. Separate L1 instruction and data caches remove this particular port conflict, but misses can still contend deeper in a unified cache/memory hierarchy.

Other structural limits include:

- one integer divider serving multiple operations;
- too few register-file ports;
- one load/store pipeline with several memory operations ready;
- limited issue, decode, or retirement width;
- finite queue, TLB, miss-status, or store-buffer entries.

“The processor has an ALU” is not enough information. Throughput depends on how many relevant operations each resource can accept and complete.

---

3. RAW carries true data flow

Consider:

```
add x5, x1, x2
sub x6, x5, x3
```

The `sub` must read the value produced by the `add`. This is **read after write (RAW)**, also called a true or flow dependence.

Without special handling, `sub` reads `x5` in ID before `add` writes it in WB:

```
cycle      1    2    3    4    5    6
add        IF   ID   EX   MEM  WB
sub             IF   ID   EX   MEM  WB
                        needs x5
```

Reading the stale register-file value would violate the program.

Forwarding uses the value when it exists

The `add` result exists at the end of its EX work. In the next cycle it is held in EX/MEM while `sub` enters EX. A bypass network can select that newer value instead of the stale ID/EX operand:

```
EX/MEM ALU result ─┐
MEM/WB result ─────┼→ forwarding mux → EX operand input
ID/EX operand ─────┘
```

Control compares source and destination identities:

```
if older instruction will write rd
and rd != x0
and younger EX source register == rd
then select the newest matching forwarded value
```

The newest match must win if several older instructions target the same register.

Forwarding does not move a value backward in time. It works only after a producer has generated the value and before a consumer needs it.

---

4. Why a load-use pair still stalls

Consider:

```
lw  x7, 0(x6)
add x8, x7, x5
```

The load computes its address in EX but receives data from memory near the end of MEM. The following `add` needs `x7` at the beginning of its EX work in that same cycle:

```
cycle      1    2    3    4    5    6    7
lw         IF   ID   EX   MEM  WB
add             IF   ID   --   EX   MEM  WB
                        stall
```

The value arrives too late for ordinary EX-to-EX forwarding. The hazard unit:

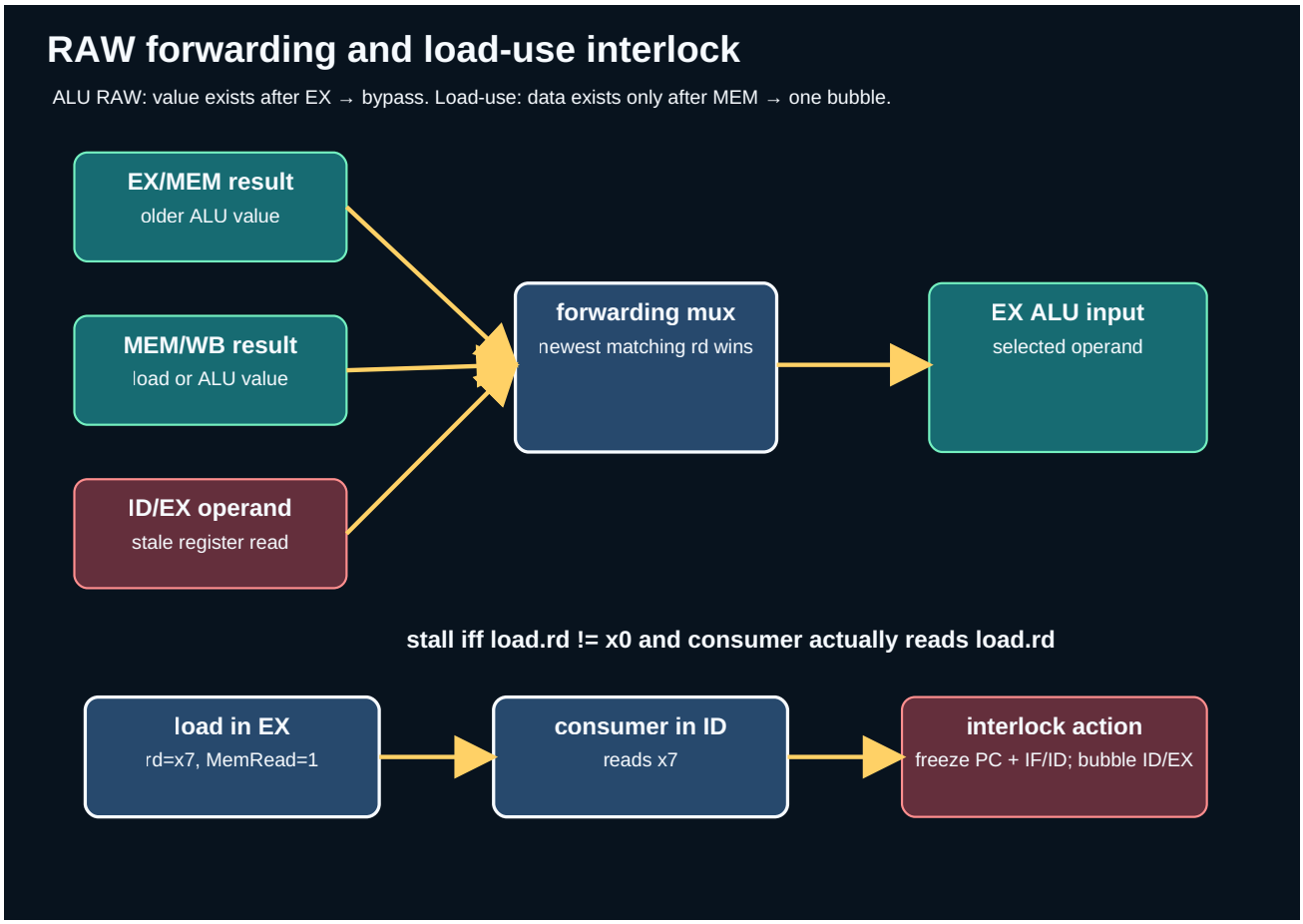
1. holds PC/fetch state;
2. holds IF/ID, keeping the consumer in ID;
3. inserts a bubble into ID/EX;
4. lets the load advance;
5. forwards the load result when the consumer reaches EX one cycle later.

In logic-like notation:

```
stall if ID/EX is a load
      and ID/EX.rd != x0
      and IF/ID actually reads ID/EX.rd
```

“Actually reads” matters. An instruction field occupying the same bit position as `rs2` is not necessarily a source register for every opcode. Hazard detection follows decoded semantics, not raw bit coincidence.

Some pipelines expose different load latencies or schedule assumptions. One stall is the result for this teaching model, not a universal law for every core.



Forwarding selects an existing newer value; the load-use interlock waits until data exists.

---

5. WAR and WAW: real names, absent timing hazards here

Three dependence names are often memorized too broadly.

RAW — read after write

```
add x5, x1, x2     # writes x5
sub x6, x5, x3     # reads x5
```

The second instruction needs a value from the first. Register renaming cannot remove this true dependence.

WAR — write after read

```
sub x6, x5, x3     # reads x5
add x5, x1, x2     # later writes x5
```

WAR is an anti-dependence: the younger write must not occur before the older read.

WAW — write after write

```
add x5, x1, x2     # older write x5
sub x5, x3, x4     # younger write x5
```

WAW is an output dependence: the final visible value must come from the younger instruction.

The important nuance

In this single-issue, in-order five-stage pipeline:

- instructions enter stages in order;
- register reads occur in ID in order;
- register writes occur in WB in order.

Therefore WAR and WAW names exist in the instruction stream, but their timing cannot invert in this pipeline. They require no special stalls. RAW can still need forwarding or a stall because a younger read naturally occurs before an older WB.

Out-of-order execution changes the timing. Register renaming maps architectural names to different physical destinations, removing false WAR and WAW dependencies while preserving RAW flow.

Do not say “RISC-V has no WAR hazards.” The ISA does not choose an execution schedule. Say: “this in-order pipeline cannot violate WAR or WAW ordering.”

---

6. Stalling must freeze the right state

A stall is not “stop the whole CPU.” For the load-use case:

```
older load:       continues EX → MEM → WB
dependent in ID:  remains in ID
instruction in IF: remains in IF
EX input:         receives a bubble
```

If PC advanced while IF/ID remained held, an instruction could be skipped. If the ID instruction advanced while its operand was stale, it could compute the wrong result. If the load were also frozen unnecessarily, the needed value would never approach availability.

The valid bit makes the bubble explicit:

```
valid=1  stage contains a real instruction
valid=0  stage contains no instruction; control effects are disabled
```

Data bits in an invalid stage may contain old electrical values. Consumers must honor validity rather than interpreting those bits.

7. Branches create control hazards

For:

```
beq x5, x0, target
```

the fetch unit needs a next PC before this teaching pipeline knows the branch outcome in EX.

Possible policies include:

```
stall until resolved      correct but wastes fetch opportunities
predict not taken         fetch sequential PC
predict taken             fetch target, if target is available
dynamic prediction       use recorded branch behavior and richer context
```

Prediction changes which work begins, not the ISA result. The machine must detect a wrong prediction and recover.

Predict not taken, actually taken

If the branch resolves in EX:

```
cycle      1    2    3    4
branch     IF   ID   EX
wrong A         IF   ID    ← flush
wrong B              IF    ← flush
```

Recovery:

1. redirect fetch to the actual target;
2. invalidate younger wrong-path instructions in IF and ID;
3. retain the branch and every older instruction;
4. prevent all killed register, memory, CSR, and exception effects.

The two visible younger instructions produce a two-slot recovery cost in this model. Deeper front ends and later resolution usually increase the amount of work at risk.

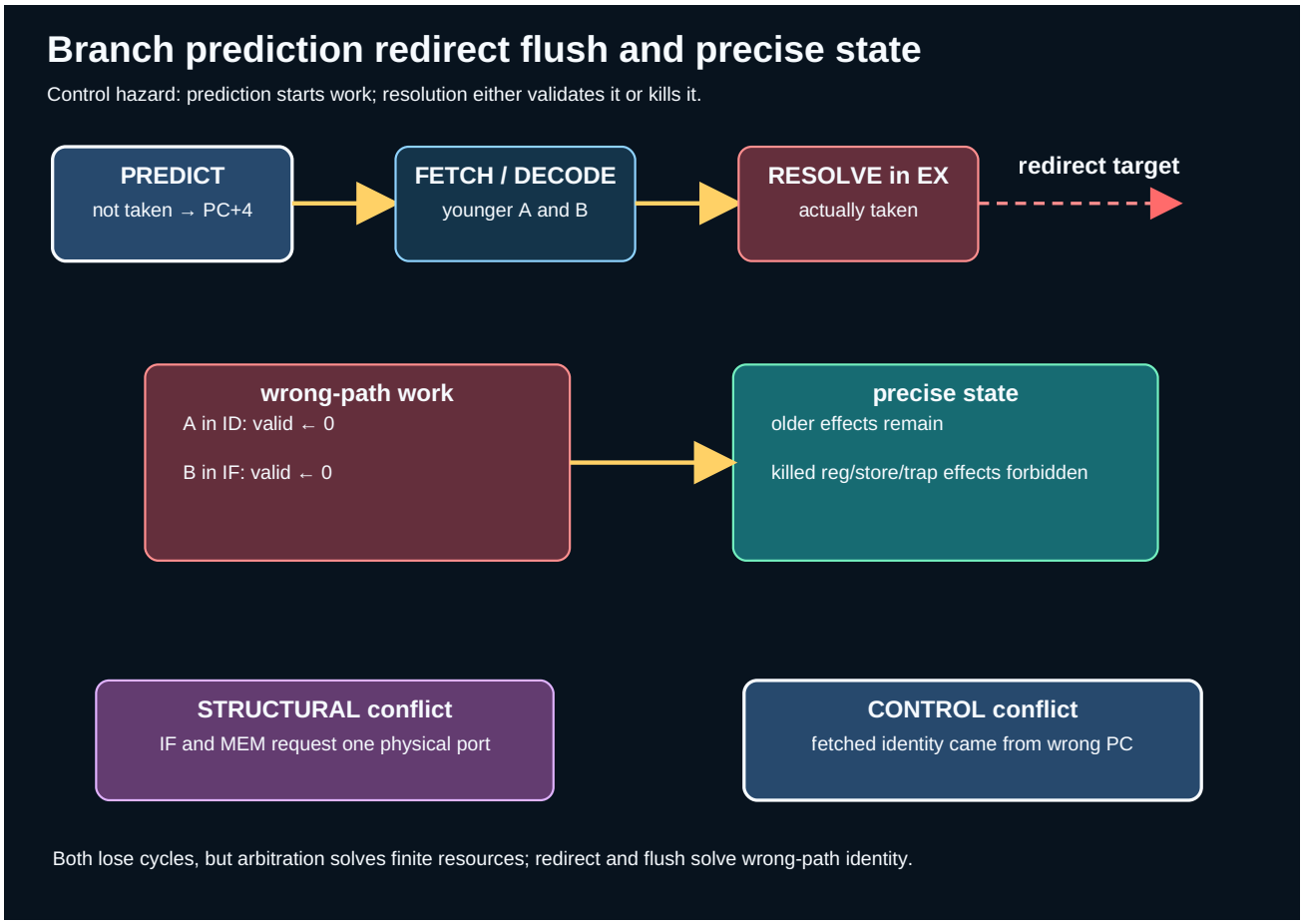
Prediction modes in the lab

The simulator supports:

- `not-taken` — always choose sequential fetch;
- `taken` — always choose the branch target;
- `one-bit` — remember the last resolved outcome for that branch PC, initialized not taken;
- `perfect` — an oracle baseline that knows the supplied outcome.

Perfect prediction is not implementable; it establishes the best-case trace. The included branch executes once, so one-bit begins like not-taken. A loop would reveal learning and its own first/last-iteration failures.

Current processors use substantially richer predictors. Their structures are microarchitectural and vendor-specific. The architectural requirement remains: wrong-path work must not alter architectural state.



A control recovery kills wrong-path identity; structural arbitration solves a different conflict.

Architecture comparison — branch dependencies and memory order

The branch's architectural inputs differ. RV32I branches compare two registers directly. AArch64 supports flag-consuming conditional branches as well as direct register zero/bit tests; x86-64 conditional branches commonly consume `RFLAGS` set by an earlier instruction. Hazard logic must therefore track the dependencies defined by its **ISA**, but predictor design, resolution stage, speculation depth, and recovery machinery remain **microarchitectural** choices on all three.

Do not extend this lesson's in-order stage schedule into a claim about memory ordering. x86-64 provides a relatively strong TSO-style architectural model but still permits effects such as store buffering. AArch64 is more weakly ordered and supplies acquire/release operations and barriers. RISC-V base systems use RVWMO, with fences and extension-dependent atomic mechanisms such as acquire/release bits, AMOs, and LR/SC. These are **architectural** constraints on observable memory behavior, not statements that a pipeline literally performs every access in source order.

Language atomics express a language-level contract; compilers select the required ISA operations, and kernels use architecture-specific barrier APIs and mapping rules as **software policy**. `volatile` alone does not establish inter-core ordering. See Architecture Comparison for Systems Programmers.

---

8. Exceptions connect flushing to precise state

A branch flush says:

```
these younger instructions were fetched from the wrong path
therefore none may commit
```

An exception needs a closely related rule:

```
older instructions may complete
the faulting instruction reports the trap
younger instructions must not commit
```

Pipeline registers therefore carry exception identity and metadata alongside results. A fault detected early may travel until the machine can establish an in-order architectural boundary.

Examples include:

- instruction-access fault in IF;
- illegal instruction in ID;
- arithmetic or privilege-related exception in EX;
- load/store page or access fault in MEM.

Stores make precision especially important

A wrong-path register result can be blocked at WB. A store that updates memory too early is harder to undo. Implementations delay or buffer stores until their architectural legitimacy is established. Exact mechanisms vary.

Interrupts

An interrupt is asynchronous to the instruction stream, but the architecture still defines where it is taken and what restart state software observes. The kernel depends on that contract to save a task, run a handler, and resume.

Virtualization

A guest page fault, privileged operation, or external event can cause a trap or VM exit. Hardware presents defined architectural state to the hypervisor. Wrong-path pipeline work is not guest-visible state. Precise boundaries make instruction emulation and reliable guest resumption possible.

---

9. CPI reasoning: account for empty retirement slots

For a long single-issue run:

```
CPIideal ≈ 1
```

A first accounting model is:

```
CPI ≈ 1
    + data-stall cycles / retired instructions
    + structural-stall cycles / retired instructions
    + control-recovery cycles / retired instructions
    + other lost cycles / retired instructions
```

This is bookkeeping, not a claim that all penalties are statistically independent.

Load-use contribution

If 20% of instructions are loads, 25% of those have an immediate dependent consumer, and each such pair costs one cycle:

```
load-use penalty/instruction = 0.20 × 0.25 × 1 = 0.05
CPI contribution             = 0.05
```

Branch contribution

If 15% of instructions are branches, prediction accuracy is 92%, and each misprediction costs two cycles in this pipeline:

```
branch penalty/instruction = 0.15 × (1 - 0.92) × 2
                           = 0.024
```

With no other losses:

```
CPI ≈ 1 + 0.05 + 0.024 = 1.074
```

For `N` retired instructions:

```
execution time ≈ N × CPI × Tclock
```

A design change can lower CPI but lengthen the clock period. For example, moving branch comparison earlier may reduce flush depth while adding decode logic to a critical stage. Compare time, not CPI alone.

Why measured CPI can be much larger

The teaching terms omit:

- instruction- and data-cache misses;
- TLB misses and page walks;
- variable-latency arithmetic;
- memory ordering and store-buffer pressure;
- front-end bandwidth and decode limits;
- interrupts, context switches, and frequency changes;
- speculation and width limits in superscalar cores.

Use the simple equation to ask where cycles went, then use architecture- specific events and documentation to refine the answer.

---

10. Kernel and performance relevance

`perf stat`

On a supported Linux system:

```
perf stat -e cycles,instructions,branches,branch-misses -- command
```

From the reported events, one may calculate:

```
CPI                  = cycles / instructions
branch miss fraction  = branch-misses / branches
```

But do not multiply branch misses by this lesson's two-cycle penalty and call it a current CPU measurement. A modern core's penalty varies with where the branch is detected, front-end state, instruction availability, overlap, and microarchitecture.

Also verify:

- what each event counts on this PMU;
- whether `instructions` means architecturally retired instructions;
- whether events were multiplexed;
- user/kernel/guest/host filters;
- migration and measurement noise.

Compiler and kernel code shape

Compilers schedule independent instructions and may use conditional operations, layout, inlining, or unrolling to alter dependencies and branch behavior. Kernel hot paths care about unpredictable branches, dependent pointer chains, cache misses, and serialization, but source appearance alone does not prove a specific hardware event.

Security mitigations can intentionally constrain speculation or indirect branch prediction. Their cost depends on CPU generation, firmware, kernel configuration, workload, and mitigation mode; use current kernel and vendor documentation.

---

11. Lab — visualize hazards and test predictions

File:

```
05-riscv-cpu-microarchitecture/challenges/day-006-hazards-forwarding-stalls-and-branches.py
```

The standard-library-only simulator tracks instruction identities through IF, ID, EX, MEM, and WB. Rows show stage occupancy at the **start** of each cycle. It models:

- full ALU-result forwarding;
- the classic one-cycle load-use stall;
- optional no-forwarding RAW stalls;
- optional unified-memory IF/MEM structural conflicts;
- branch resolution in EX;

- not-taken, taken, one-bit, and perfect prediction;
- wrong-path flushing and in-order retirement counts.

It does not compute register values, model caches, or represent a specific commercial processor.

Part A — data hazards

Before running, predict the stalls in:

```
add x5, x1, x2
sub x6, x5, x3
lw  x7, 0(x6)
add x8, x7, x5
sw  x8, 8(x1)
```

Run:

```
python3 05-riscv-cpu-microarchitecture/challenges/day-006-hazards-forwarding-stalls-and-branches.py
trace data
python3 05-riscv-cpu-microarchitecture/challenges/day-006-hazards-forwarding-stalls-and-branches.py
trace data --no-forwarding
```

Explain:

1. why `add → sub` forwards without a stall;
2. why `lw → add` stalls once even with forwarding;
3. why no-forwarding mode waits longer;
4. why the store's data source is still a dependence.

Part B — branches and prediction

Run all four baselines:

```
python3 05-riscv-cpu-microarchitecture/challenges/day-006-hazards-forwarding-stalls-and-branches.py
trace branch --prediction not-taken
python3 05-riscv-cpu-microarchitecture/challenges/day-006-hazards-forwarding-stalls-and-branches.py
trace branch --prediction taken
python3 05-riscv-cpu-microarchitecture/challenges/day-006-hazards-forwarding-stalls-and-branches.py
trace branch --prediction one-bit
python3 05-riscv-cpu-microarchitecture/challenges/day-006-hazards-forwarding-stalls-and-branches.py
trace branch --prediction perfect
```

Record:

- which PCs are flushed;
- which instructions retire;
- cycles and CPI;
- why one-bit and not-taken match for a first encounter;
- why perfect prediction is evidence about an upper bound, not a design.

Part C — structural conflict

```
python3 05-riscv-cpu-microarchitecture/challenges/day-006-hazards-forwarding-stalls-and-branches.py
trace structural
python3 05-riscv-cpu-microarchitecture/challenges/day-006-hazards-forwarding-stalls-and-branches.py
trace structural --unified-memory
```

Locate every cycle where a MEM-stage load/store blocks IF.

Part D — prediction exercises and validation

```
python3 05-riscv-cpu-microarchitecture/challenges/day-006-hazards-forwarding-stalls-and-branches.py
exercise
python3 05-riscv-cpu-microarchitecture/challenges/day-006-hazards-forwarding-stalls-and-branches.py
exercise --answers
python3 05-riscv-cpu-microarchitecture/challenges/day-006-hazards-forwarding-stalls-and-branches.py
selftest
```

Write the RAW/WAR/WAW classifications before revealing answers.

---

12. Read the simulator critically

The trace begins with an empty cycle, so do not force its total into a formula using a different cycle-boundary convention. Compare traces generated by the same model and explain each visible bubble.

The simulator's retirement count excludes flushed instructions. That makes a critical point visible:

```
fetched work can consume cycles
without becoming retired architectural work
```

Its one-bit predictor has persistent state during one simulation, but the included branch is encountered only once. Change of predictor state matters only if a branch PC is fetched again.

The model also assumes:

- one instruction fetched and retired at most per cycle;
- fixed instruction latency by class;
- branch target available when predicted taken;
- register-file timing compatible with its no-forwarding stall rule;
- no cache misses or exceptions in the executable scenarios.

State these assumptions when using its output as evidence.

---

13. Evidence: Explain, Draw, Observe, Connect

Explain

Without notes:

1. distinguish structural, data, and control hazards;

2. classify RAW, WAR, and WAW;
3. explain why only RAW creates a timing hazard in this pipeline;
4. derive one ALU forwarding decision;
5. explain the load-use stall cycle;
6. explain prediction, detection, redirect, and flush;
7. connect valid/kill state to precise exceptions;
8. derive one CPI contribution from frequencies and penalties.

Draw

Draw:

```
ID/EX operand ──────┐
EX/MEM result ──────┼→ forwarding mux → ALU input
MEM/WB result ──────┘
```

Then add:

- destination/source comparisons;
- newest-match priority;
- load-use detection;
- PC and IF/ID enables;
- ID/EX bubble insertion;
- branch redirect and IF/ID kill.

Observe

Preserve one trace for each:

- forwarded ALU dependency;
- load-use stall;
- no-forwarding RAW wait;
- unified-memory structural stall;
- branch misprediction and flush;
- correct prediction.

Annotate the exact instruction identities. “There is a bubble” is not enough; say why it exists and which architectural error it prevents.

Connect

Choose one systems case:

- a page fault and restart;
- a VM exit and guest resume;
- a `perf stat` branch-miss observation;
- a kernel pointer-chasing path;
- a speculation mitigation.

Identify the architectural contract, the relevant microarchitectural mechanism, and one fact requiring current platform documentation.

14. Mastery checkpoint

Answer:

> How does an in-order five-stage processor preserve program meaning while instructions overlap, and where do lost cycles come from?

A complete answer includes:

- finite resources and structural arbitration;
- RAW detection and forwarding;
- one-cycle load-use interlock;
- why WAR/WAW do not reorder in this pipeline;
- prediction as speculative next-PC selection;
- redirect and younger-instruction flush;
- valid/kill control preventing wrong effects;
- in-order retirement and precise-state reasoning;
- CPI as retired work plus lost-cycle accounting.

Why? notebook

1. Why is a dependence not automatically a stall?
2. Why can forwarding solve `add → sub` but not eliminate time itself?
3. Why does a load's address being ready not mean its loaded data is ready?
4. Why must hazard logic know whether an encoded field is really read?
5. Why are WAR and WAW names still useful when this pipeline cannot violate them?
6. Why must a load-use stall freeze IF/ID but let the load advance?
7. Why does prediction improve throughput without changing ISA semantics?
8. Why must a wrong-path store be prevented from reaching memory?
9. Why is branch accuracy alone insufficient to calculate execution time?
10. Why can `cycles / instructions` be informative but not identify the cause of a slowdown?
11. Why does precise state matter to page faults and VM exits?
12. Why should a current CPU's penalties come from measurement and vendor documentation rather than the five-stage diagram?

References used selectively

- RISC-V International, *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture*, current control-flow, load/store, and register semantics: https://docs.riscv.org/
- RISC-V International, *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture*, current trap, interrupt, and virtualization contracts: https://docs.riscv.org/

- David A. Patterson and John L. Hennessy, *Computer Organization and Design: RISC-V Edition*, processor chapter, for the pedagogical five-stage hazard, forwarding, stall, and branch model.
- Linux `perf-stat(1)` manual, current event selection, filtering, multiplexing, and metric behavior: https://man7.org/linux/man-pages/man1/perf-stat.1.html
- Linux kernel documentation, "Hardware vulnerability and mitigation information," for current kernel-visible speculation mitigations: https://docs.kernel.org/admin-guide/hw-vuln/
- Intel, *64 and IA-32 Architectures Optimization Reference Manual*, current dependency, branch, memory, retirement, and PMU guidance: https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html
- AMD, *Processor Programming Reference and Software Optimization Guides*, current family-specific pipeline and PMU guidance: https://www.amd.com/en/search/documentation/hub.html
- Arm, *Performance Monitoring Unit architecture* and Statistical Profiling Extension documentation, for current event and attribution contracts: https://developer.arm.com/Architectures
- Linux KVM documentation, for current guest/host architectural-state and virtualization interfaces: https://docs.kernel.org/virt/kvm/

Next bridge: caches and the memory hierarchy turn the fixed MEM stage into a variable-latency system. Miss handling, TLB translation, ordering, and memory-level parallelism extend the same resource, dependency, and retirement questions.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Hazards Forwarding Stalls And Branches

Format: Python lab

What you will do: Visual five-stage hazard simulator and prediction exercises.

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
python3 05-riscv-cpu-microarchitecture/challenges/day-006-hazards-forwarding-stalls-and-branches.py
```

- View lab source on GitHub

Day 7 — Cache, TLB, MMU, and End-to-End Task Trace

Target time: approximately 2½–3 hours

- Core-to-cache requests, hits, misses, and stalls: 40 minutes
- TLB translation, page-table walks, and permission checks: 45 minutes
- Runnable observation model: 40 minutes
- C `sum` trace from source to transistor switching: 45 minutes
- Mastery evidence and systems bridge: 20 minutes

> The core has decoded a load from virtual address `0x4004`. Why can it not simply read that address from RAM and continue on the next clock?

Three independent questions must be answered:

1. **Translation:** which physical address, if any, corresponds to this virtual address?
2. **Permission:** may this access execute, read, or write at the current privilege level?
3. **Data availability:** does the relevant cache already hold the physical line, or must the request wait for a lower level?

A TLB helps answer the first two questions quickly. A cache helps answer the third. A TLB miss is not a cache miss, and neither event is automatically a fault.

Today ends the CPU-design module by tracing a small task through every layer:

```
C source
  → compiler-selected RISC-V instructions
  → architectural effects
  → fetch/decode/datapath activity
  → cache and TLB behavior
  → OS-created page-table policy
  → gates, wires, and state elements
  → transistor conduction and charge movement
```

---

1. Four layers, four kinds of statement

The most important discipline is to label each claim.

Architectural layer: the software-visible contract

For this lesson, architectural facts include:

- the meanings of RISC-V instructions such as `lw`, `add`, `addi`, and `bne`;
- the values eventually visible in integer registers;
- the virtual address generated by a load or instruction fetch;
- whether an access completes or raises an architectural exception;
- the required ordering behavior of memory operations.

The ISA does not require a particular cache size, TLB replacement rule, pipeline depth, or transistor arrangement.

Microarchitectural layer: how this core implements the contract

Implementation choices include:

- separate L1 instruction and data caches;
- direct-mapped versus set-associative caches;
- whether cache lookup overlaps address translation;
- TLB size and replacement;
- a hardware page-table walker;
- request/response handshakes, miss-status state, and pipeline stalls;
- coherence protocol and ordering machinery.

Two compliant cores may choose differently and still run the same binary.

OS-policy layer: what mappings and permissions should exist

The operating system:

- allocates physical page frames;
- creates and changes page-table entries;
- assigns user/supervisor and read/write/execute permissions;
- chooses address-space identifiers and mappings;
- handles page-fault exceptions;
- decides whether to allocate, load, kill, signal, or resume a task.

Hardware enforces encoded permissions. It does not decide that a C array “deserves” another page.

Physical layer: how bits actually change

At the physical layer:

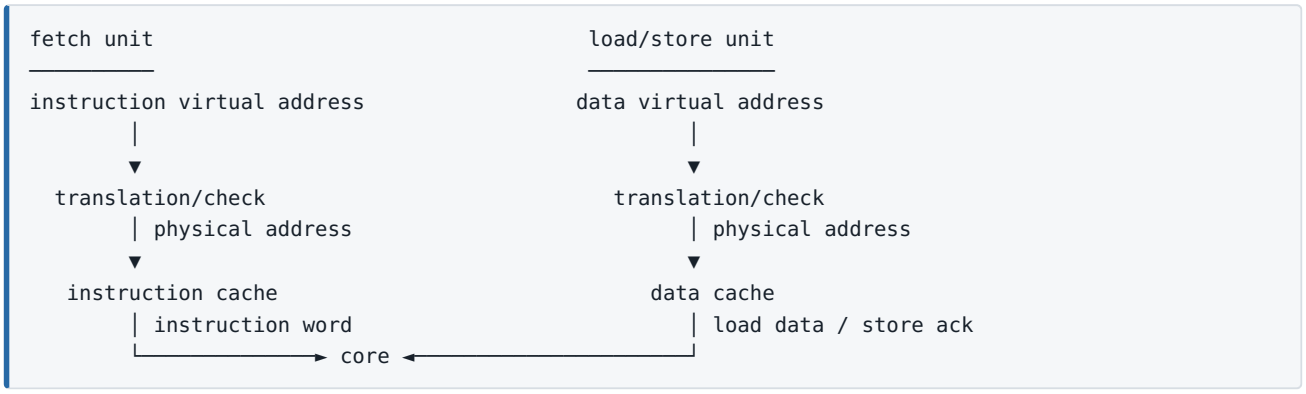
- voltage ranges represent bits;
- transistor gates control conductive paths;
- capacitances charge or discharge;
- combinational paths settle after finite delay;
- clocked state captures values subject to setup/hold constraints;
- switching consumes energy and produces heat.

“The TLB compares a tag” is a microarchitectural statement. Physically, that comparison is a large network of switching devices and wires.

---

2. Connecting the core to instruction and data caches

A simple core can expose two logical request paths:



Separate paths permit an instruction fetch and data access to proceed together. A unified cache is also possible. Neither organization is mandated by the ISA.

A request needs backpressure

The core cannot assume every access finishes immediately. A useful conceptual interface is:

```
request:  valid, address, operation, size, write_data
response: ready, read_data, fault
```

On a hit, `ready` may return quickly. On a miss, the cache records enough information to request the line from the next level. The pipeline stage holding the dependent instruction must wait, or another mechanism must track the outstanding work.

For a small in-order core:

```
data-cache hit
  → load obtains bytes
  → extension/writeback proceeds

data-cache miss
  → load cannot retire yet
  → freeze dependent pipeline state
  → request line from lower level
  → fill cache
  → replay or complete load
  → resume
```

A **stall** changes when progress occurs, not what the instruction means. The implementation must hold or explicitly invalidate each affected state element. Merely “pausing the PC” while later stages continue to overwrite pipeline registers corrupts instruction ownership.

Instruction misses also stall progress

If the instruction cache misses, no valid instruction word exists for that PC yet. A small core can hold the fetch PC and inject no new instruction until the line arrives. Older instructions already in the pipeline may still finish if the design permits it.

Real cores can sustain several misses, speculate, prefetch, execute unrelated instructions, and retire in order. Those optimizations enlarge the tracking problem; they do not remove the need to preserve precise architectural behavior.

---

3. A tiny direct-mapped cache model

Use a deliberately small cache:

```
line size = 16 bytes
sets      = 4
ways      = 1
capacity  = 64 bytes
```

For physical address `PA`:

```
line_number = PA // 16
offset      = PA % 16
set         = line_number % 4
tag         = line_number // 4
```

Each set stores a valid bit, tag, and sixteen data bytes. On lookup:

1. select one set;
2. compare its stored tag while checking valid;
3. on a match, select bytes by offset;
4. otherwise fetch the aligned physical line and replace that set.

The model below charges one cycle for a hit and eight extra cycles for a fill. Those numbers are teaching parameters, not claims about RISC-V or real silicon.

Below the cache: one bounded physical path

A cache lookup commonly reads SRAM-like tag and data arrays: address bits select entries, tag-compare logic determines a hit, and data-selection logic returns the requested bytes. Custom cache arrays need not be built like the Day 2 RTL register file, and this organization is a microarchitectural choice, not an ISA property.

On a miss, one representative path is:

```
physical cache-line request
  → lower cache/interconnect, if present
  → memory controller schedules a DRAM command sequence
  → activate a bank row
  → sense amplifiers resolve small bitline changes and restore row data
  → transfer selected columns as cache-line beats
  → precharge/equalize the bank for a later row
  → return beats through the controller
  → fill cache data/tag arrays and mark the line valid
  → select the requested load bytes and wake the waiting core operation
```

Activation raises a row's wordline and connects many DRAM cells to bitlines. Sense amplification turns the resulting small voltage differences into logic levels; because the read disturbs cell charge, sensing also restores the row. Precharge prepares the bitlines before another row is activated. Separately, periodic refresh must revisit rows before charge decays too far and can delay demand traffic. Open-row hits, controller scheduling, bank parallelism, lower cache levels, channel width, and exact command timing vary by system; not every miss performs an immediate precharge in exactly this order.

The controller, queues, cache-fill state, and command policy are microarchitecture. Wordlines, bitlines, sense amplifiers, charge leakage, restoration, and switching are circuit/physical mechanisms. The architectural fact is only that the load eventually returns the required value or reports an exception; software does not observe “row activation” as part of `LW`.

Why caches normally use physical identity

Virtual aliases can map different virtual addresses to the same physical page. If a cache treated those aliases as unrelated storage, the core could hold inconsistent copies of one physical location. Practical designs use physical tags or carefully constrained virtual indexing and OS cooperation.

The exact point at which translation and cache lookup overlap is microarchitectural. Keep the correctness model simple:

```
virtual request → translate/check → physical cache lookup
```

---

4. Translation: page offset stays, page identity changes

Assume 4 KiB pages:

```
virtual address
┌──────────────── virtual page number ────────────────┬─ page offset ─┐
│                       VPN                           │    12 bits    │
└─────────────────────────────────────────────────────┴───────────────┘
```

A valid page-table entry supplies a physical page number:

```
physical address = (PPN << 12) | page_offset
```

Translation changes the page identity, not the offset within that page.

What the TLB caches

A translation lookaside buffer caches recently used translation metadata:

```
(address-space identity, virtual page number)
    → physical page number + permission/state attributes
```

It does not cache the array element's value. That is the data cache's job. It also must not confuse identical virtual addresses belonging to different process address spaces; an ASID or TLB invalidation strategy handles that distinction.

TLB hit

On a matching valid TLB entry:

1. retrieve PPN and permission attributes;
2. check the requested access and privilege;
3. form the physical address;
4. continue toward the cache.

TLB miss

A TLB miss means only:

> The fast translation cache lacks the needed translation.

On systems with a hardware page-table walker, the MMU reads page-table structures from memory. Those reads can themselves interact with caches. If the walk finds a valid permitted leaf entry, the hardware can fill the TLB and retry the original access.

```
TLB miss
  → walker locates root selected for this address space
  → reads page-table entry at each required level
  → validates leaf and permissions
  → fills TLB
  → retries original access
```

The full RISC-V Sv32/Sv39 algorithms include levels, PTE formats, superpages, accessed/dirty state, and precise fault conditions. The lab compresses these details into a dictionary lookup so the responsibility boundary remains visible. It is not an Sv39 emulator.

Page-table memory is ordinary memory with an extraordinary role

A page table is a data structure in memory created by privileged software. The CPU is told where its root is and interprets entries according to the architecture. A hardware walk therefore creates extra memory traffic:

```
one requested load
  → zero page-table reads on a TLB hit
  → several possible page-table reads on a TLB miss
  → then the requested data-cache access
```

This is why TLB reach and page-walk locality affect system performance.

---

5. Missing, invalid, and denied are not the same

Suppose a walk reaches one of these simplified entries:

```
VPN 0x4 → PPN 0x20, user, readable, not writable, not executable
VPN 0x1 → PPN 0x10, user, executable, readable
VPN 0x9 → no valid entry
```

Results:

- user instruction fetch from VPN `0x1`: translation can succeed;
- user load from VPN `0x4`: translation can succeed;
- user store to VPN `0x4`: permission fault;
- access to VPN `0x9`: not-present/invalid translation fault.

A cache hit cannot override a permission failure. Permission checking belongs to translation/protection and must occur even if physical data remains in a cache from earlier use.

Fault mechanism versus OS response

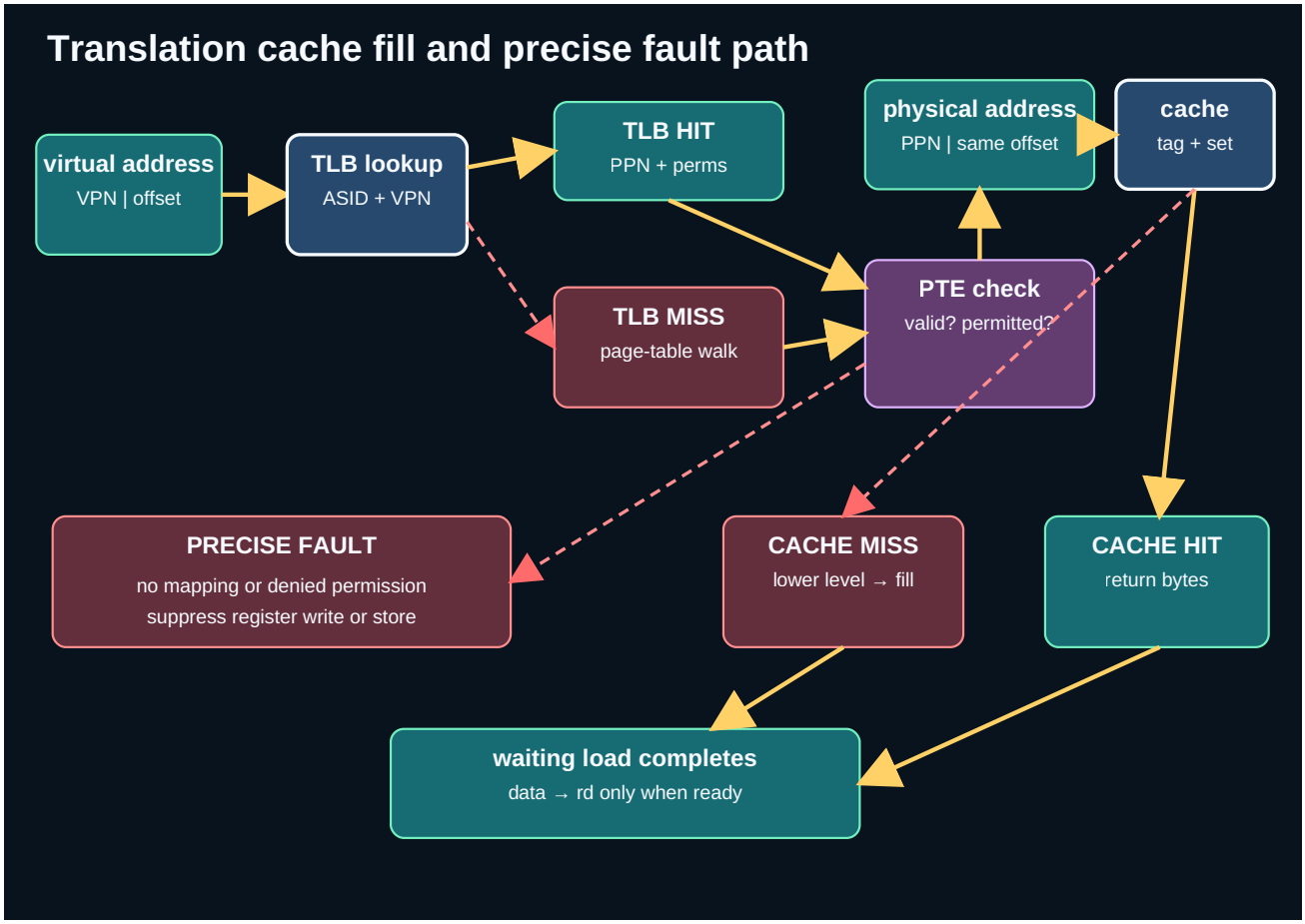
The hardware:

1. detects that the architectural access cannot complete;
2. records a cause and faulting virtual address;
3. preserves a precise restart point according to the ISA;
4. transfers control to a privileged trap handler.

The OS then applies policy:

- map a page and resume for a legitimate demand fault;
- update bookkeeping and resume where architecture permits;
- reject an illegal access and terminate or signal the process;
- emulate or otherwise handle a special mapping.

“The MMU kills the process” is wrong. The MMU reports an architectural fault; the OS decides the task-level response.



TLB and cache misses may wait and recover; denied or absent translations take the precise fault path.

---

6. Ordering and coherence: boundary map only

Today needs two definitions, not a complete protocol or memory-model course.

Memory ordering

Memory ordering specifies which observations of loads, stores, and explicit ordering operations software may rely on. A core may buffer, overlap, or reorder internal memory work, but it must obey the architecture's ordering rules. RISC-V has a relatively weak memory model and provides mechanisms such as `FENCE`; language atomics and synchronization libraries compile to the required instruction sequences.

A cache hit does not by itself prove that all cores may observe earlier stores in source order.

Cache coherence

Coherence coordinates copies of the same physical cache line among agents so writes and ownership are handled consistently. It operates on physical memory identity and commonly at cache-line granularity. Coherence does not replace:

- page-table permissions;
- the architectural memory-ordering model;
- C's data-race and atomic rules;
- OS synchronization policy.

For this tiny single-core observation model, coherence is intentionally out of scope. The model has only one cache hierarchy and no second observer.

Architecture comparison — memory systems, privilege, and virtualization

Across x86-64, AArch64, and RISC-V, cache and TLB capacities, associativity, replacement, split/unified organization, and walk-cache design are **microarchitectural**. The **architecture** instead defines translation formats, permissions, faults, maintenance operations, privilege state, and memory-ordering guarantees:

- x86-64 commonly uses `CR3`-rooted multilevel tables, 4 KiB base pages and larger pages; supported address widths and level counts are feature dependent. Privilege is expressed through CPL/rings and page permissions. VMX/SVM systems can add EPT/NPT second-level translation.
- AArch64 uses translation regimes selected through system registers, with architected 4 KiB, 16 KiB, and 64 KiB granules. EL0–EL3 describe exception levels; EL2 and stage-2 translation support virtualization.
- RISC-V selects a supported scheme through `satp`: RV32 commonly uses Sv32, while RV64 systems may implement Sv39, Sv48, or Sv57. U/S/M modes define privilege; the optional hypervisor extension adds HS/VS/VU state and two-stage translation.

Page-table entry layouts, fault reports, invalidation instructions, and virtualization control state therefore differ. All three still need translation caching, permission checks, precise faults, and synchronization after mapping changes. Their ordering contracts also differ: x86-64 is relatively strong and TSO-style, while AArch64 and RISC-V RVWMO permit weaker ordering with explicit acquire/release and barrier/fence mechanisms.

The **OS or hypervisor policy** remains distinct: software allocates page tables, chooses mappings and permissions, handles faults, performs required TLB maintenance, and decides whether a guest or task resumes. Linux and KVM provide common abstractions, but architecture-specific code supplies these mechanisms. See Architecture Comparison for Systems Programmers.

---

7. Runnable safe observation lab

This lab models:

- 4 KiB virtual pages;
- a four-entry fully associative TLB;
- an OS-supplied page-table dictionary;
- translation and permission checks;
- separate tiny direct-mapped instruction and data caches;
- explicit cycle charges for hits, cache fills, and page-table walks.

It reads no privileged state, changes no system mappings, requires only Python 3, and writes nothing. Run from any directory:

```
python3 - <<'PY'
from dataclasses import dataclass

PAGE = 4096
LINE = 16
SETS = 4

@dataclass(frozen=True)
class PTE:
    ppn: int
    user: bool
    read: bool
    write: bool
    execute: bool

PAGE_TABLE = {
    0x1: PTE(0x10, True, True,  False, True),   # code: VA 0x1000
    0x4: PTE(0x20, True, True,  False, False),  # array: VA 0x4000
    0x7: PTE(0x30, True, True,  True,  False),  # writable data
}

class TLB:
    def __init__(self, capacity=4):
        self.capacity = capacity
        self.entries = {}
        self.order = []

    def translate(self, va, access):
        vpn, offset = divmod(va, PAGE)
        if vpn in self.entries:
            pte = self.entries[vpn]
            print(f"  [microarchitecture] TLB HIT  vpn=0x{vpn:x}")
            cycles = 1
        else:
            print(f"  [microarchitecture] TLB MISS vpn=0x{vpn:x}")
            print("  [microarchitecture] hardware page-table walk")
            pte = PAGE_TABLE.get(vpn)
            cycles = 6
            if pte is None:
                raise PermissionError(
                    f"[architecture] page fault: no valid PTE for VA 0x{va:x}"
                )
            if len(self.order) == self.capacity:
                victim = self.order.pop(0)
                del self.entries[victim]
            self.entries[vpn] = pte
            self.order.append(vpn)
            print(
                f"  [OS policy data] PTE vpn=0x{vpn:x} -> ppn=0x{pte.ppn:x}"
            )

        allowed = {
            "fetch": pte.execute,
            "load": pte.read,
            "store": pte.write,
        }[access]
        if not pte.user or not allowed:
            raise PermissionError(
                f"[architecture] {access} page fault at VA 0x{va:x}"
            )
        return pte.ppn * PAGE + offset, cycles
```

```
class DirectCache:
    def __init__(self, name):
        self.name = name
        self.lines = [None] * SETS

    def access(self, pa):
        line_number = pa // LINE
        index = line_number % SETS
        tag = line_number // SETS
        hit = self.lines[index] == tag
        event = "HIT " if hit else "MISS"
        print(
            f"  [microarchitecture] {self.name} {event} "
            f"pa=0x{pa:x} set={index} tag=0x{tag:x}"
        )
        if not hit:
            self.lines[index] = tag
        return 1 if hit else 9

tlb = TLB()
icache = DirectCache("I-cache")
dcache = DirectCache("D-cache")
cycles = 0

def request(kind, va):
    global cycles
    print(f"\n[architecture] {kind} VA 0x{va:x}")
    try:
        pa, cost = tlb.translate(va, kind)
        print(f"  [architecture] translated PA 0x{pa:x}")
        cost += (icache if kind == "fetch" else dcache).access(pa)
        cycles += cost
        print(f"  [microarchitecture] request completes; +{cost} cycles")
    except PermissionError as fault:
        cycles += 6
        print(f"  {fault}")
        print("  [OS policy] trap handler decides whether task may resume")

for kind, va in [
    ("fetch", 0x1000),  # TLB miss, I-cache miss
    ("fetch", 0x1004),  # TLB hit, same I-cache line hit
    ("load",  0x4000),  # TLB miss, D-cache miss
    ("load",  0x4004),  # TLB hit, same D-cache line hit
    ("store", 0x4004),  # TLB hit, denied by read-only PTE
    ("load",  0x9000),  # TLB miss, no valid PTE
]:
    request(kind, va)

print(f"\n[observation] modeled total = {cycles} cycles")
print("[limit] timing and policies are teaching parameters, not a real CPU")
PY
```

Predict before running

For each request, write:

1. VPN and page offset;
2. whether the TLB should hit;
3. whether translation is permitted;

4. physical address when permitted;
5. cache set and expected hit/miss;
6. whether control returns normally or enters the trap path.

Explain after running

Account for these distinct events:

- The first fetch misses in both the TLB and I-cache.
- The second fetch hits in the TLB and the same 16-byte cache line.
- The first array load requires a walk and a D-cache fill.
- The second array load uses both cached translation and cached data line.
- The store faults even though the D-cache has the line.
- The final load faults because no valid mapping exists.

Then change only one item:

- grant write permission to VPN `0x4`;
- move the second load to `0x4010`, the next cache line;
- reduce TLB capacity to one;
- add a valid PTE for VPN `0x9`.

Predict which layer's events change. Do not describe a permission change as a cache optimization or a cache-line change as OS policy.

What this lab cannot prove

It is not cycle-accurate and does not implement:

- the full RISC-V privileged architecture or Sv32/Sv39 PTE format;
- multilevel page-table address calculation;
- real cache data bytes, dirty writeback, or nonblocking misses;
- accessed/dirty-bit rules, superpages, ASIDs, or TLB shootdowns;
- speculation, prefetching, coherence, or the RISC-V memory model;
- physical timing, power, analog behavior, or transistor count.

Its evidence is narrower: the event ordering and layer boundaries are explicit and mechanically repeatable.

---

8. Compile a tiny `sum` into architectural work

Use:

```
int sum(const int *a, int n) {
    int s = 0;
    for (int i = 0; i < n; ++i)
        s += a[i];
    return s;
}
```

A compiler targeting RV32I might choose a loop conceptually like this:

```
# a0 = pointer a, a1 = n, return value in a0
sum:
    addi t0, x0, 0       # s = 0
loop:
    bge  x0, a1, done    # if n <= 0, finish
    lw   t1, 0(a0)       # load *a
    add  t0, t0, t1      # s += *a
    addi a0, a0, 4       # a++
    addi a1, a1, -1      # n--
    jal  x0, loop
done:
    addi a0, t0, 0       # return s
    jalr x0, 0(ra)
```

This is illustrative, not a promise of exact compiler output. Optimization level, extensions, ABI assumptions, alignment knowledge, unrolling, and vector support can produce different legal sequences. The C abstract machine defines behavior; the compiler chooses instructions that preserve that behavior under its assumptions. The concrete trace assumes `a` designates at least `n` valid `int` objects and the mathematical sum is representable as an `int`; otherwise the C source itself may have undefined behavior.

One iteration, layer by layer

Assume `a0 = 0x4000`, `a1 = 2`, and memory contains `7, 11`.

1. Architectural view

For `lw t1, 0(a0)`:

```
effective virtual address = x[a0] + sign_extend(0) = 0x4000
architectural result      = x[t1] receives the loaded 32-bit value
```

If translation or permission fails, the register write must not commit.

2. Fetch microarchitecture

The fetch unit presents the instruction PC as a virtual address. The instruction TLB translates it; the I-cache supplies the instruction word or stalls fetch during a fill. The PC and pipeline-valid state are held or redirected according to the design.

3. Decode microarchitecture

Decoder logic recognizes the `lw` opcode and fields. It selects:

- register `a0` as source;
- an I-type immediate of zero;
- addition in the address-generation ALU;
- a four-byte load;
- writeback to `t1`;
- no store and no branch redirect.

These control wires are not architectural registers. They are one implementation's way of producing the required effect.

4. Datapath and translation

The register file drives `0x4000`. The immediate network drives zero. Operand muxes select them, and the adder settles to virtual address `0x4000`.

The data TLB either supplies the PPN and permissions or triggers a page-table walk. The PTE exists because the OS mapped the process's array page. If it is valid, user-readable, and sufficiently aligned for the architecture's rules, the MMU forms the physical address.

5. Data cache and stall

On a D-cache hit, the selected bytes proceed toward load-data alignment and writeback. On a miss:

```
hold the load and its destination identity
  → request the physical line
  → wait
  → fill
  → obtain the four bytes
  → write t1 only when the instruction completes
```

The ISA-visible result is still `t1 = 7`; only elapsed cycles differ.

6. Dependent addition

`add t0, t0, t1` depends on the load. If data returns late, forwarding cannot invent it. An in-order pipeline stalls until the value exists, then operand muxes select the old sum and loaded value. The ALU computes the new sum.

After two iterations:

```
t0 = 18
a0 = 0x4008
a1 = 0
```

The branch reaches `done`, and ABI-selected register `a0` receives 18.

---

9. Continue below RTL: gates and transistor switching

Choose one bit slice of the address addition or sum addition.

At the logical level, a full-adder relation is:

```
sum_i   = A_i XOR B_i XOR carry_i
carry_o = (A_i AND B_i) OR (carry_i AND (A_i XOR B_i))
```

The physical story is:

1. register output nodes hold voltage ranges representing operand bits;
2. decode and select signals drive mux gate inputs;
3. transistor networks in those muxes establish conductive pull-up or pull-down paths;
4. effective node capacitances charge or discharge;

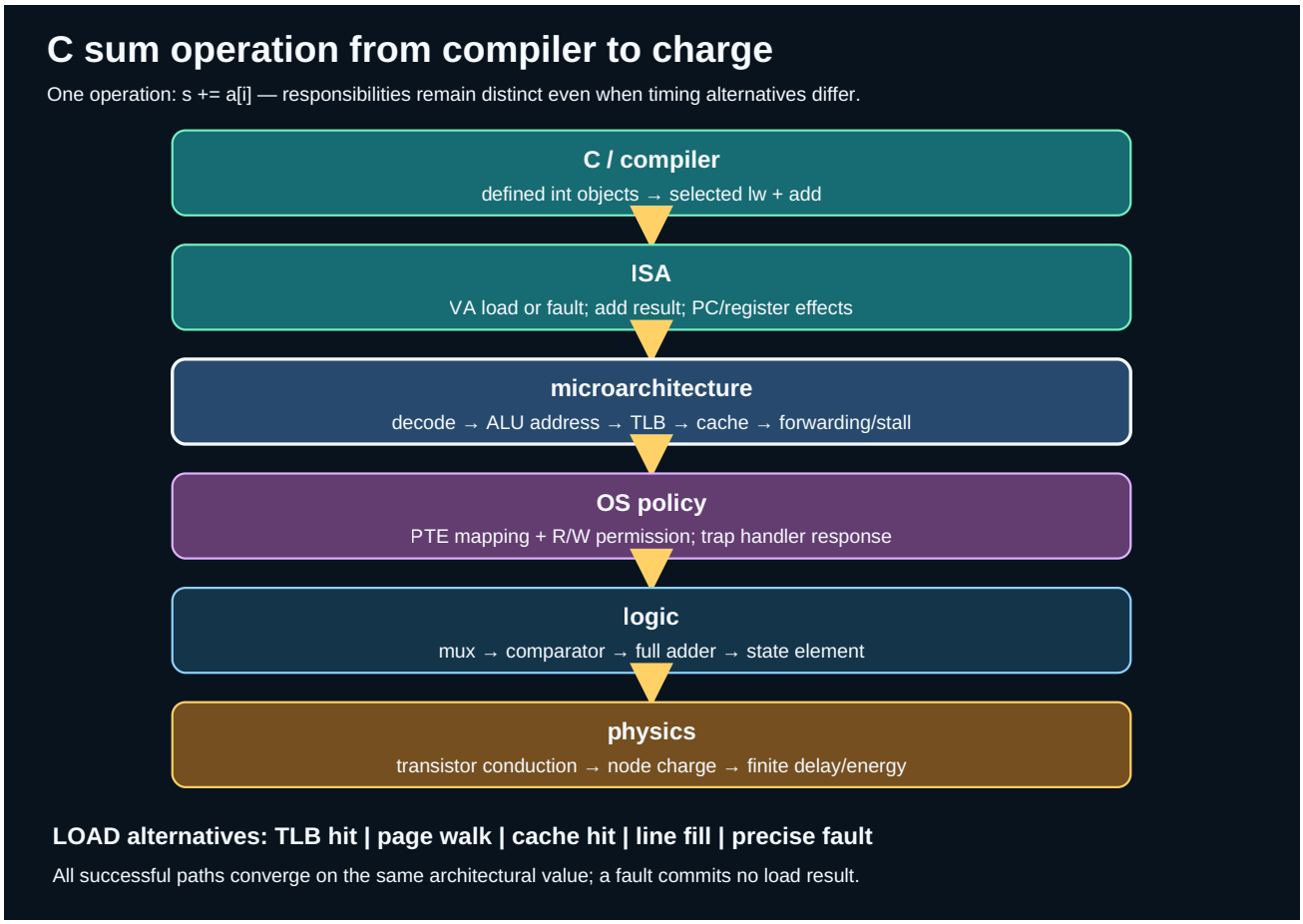
5. changed voltage ranges propagate through the adder's gate network;
6. carry and sum nodes settle after finite delay;
7. a destination state element samples its input at an allowed clock edge;
8. clock distribution and state-element switching consume energy even when the architectural instruction name says only `add`.

Do not claim that one C `+` maps permanently to one physical adder. The compiler may transform the loop; the microarchitecture may share, pipeline, or duplicate arithmetic; synthesis may rewrite the gate network; physical implementation chooses concrete cells and wires.

The complete trace

```
C: s += a[i]
│
├─ language/compiler layer
│    defined C values and compiler-selected lw/add instructions
│
├─ architectural layer
│    PC, registers, virtual load address, result, possible exception
│
├─ microarchitectural layer
│    fetch/decode, register read, ALU, pipeline state, I/D cache, TLB,
│    page-table walker, stalls, fills, forwarding, writeback
│
├─ OS-policy layer
│    page-table root, VA→PA mapping, user/read/write/execute permissions,
│    fault-handler decision
│
└─ physical layer
     voltage ranges, transistor conduction, capacitance, delay, clocked
     capture, switching energy
```

The causal chain runs downward to physics and back upward through committed architectural state. No layer makes the others unnecessary.



Successful hit and fill paths converge on one ISA result; a fault commits no load value.

End-to-end evidence workflow using existing artifacts

This is a correlation workflow, not a new lab and not a formal equivalence proof:

1. **Fix the software artifact.** Reuse Week 3's source-to-object workflow and ELF/loading workflow. Record the source, compiler options, ELF `Machine`, symbol, section bytes, and disassembly around one selected C operation. `readelf` proves metadata encoded in that ELF; `objdump` proves its decoding of those bytes for the file's declared architecture. Existing host-architecture disassembly must not be presented as RV32I machine code.
2. **State the compiler correlation carefully.** Use source locations, symbols, and dataflow to identify the instruction or sequence implementing the operation. Optimized C need not map one statement to one instruction, and disassembly alone does not prove runtime execution.
3. **Trace architectural and microarchitectural effects.** Reuse the Week 5 tiny-core trace for tested PC/register/memory transitions and this day's cache/TLB trace for modeled translation, permission, hit/miss, fill, stall, and fault events. These Python traces prove behavior of their teaching models for the exercised inputs, not of the ELF artifact or a fabricated CPU.
4. **Observe RTL state and synthesis.** Reuse Week 4's HDL/waveform evidence to show tested combinational outputs and edge-triggered capture, then its Yosys evidence to show how the selected RTL is transformed into a netlist under recorded synthesis settings. Simulation does not prove physical timing; synthesis does not prove that separate Week 5 models are equivalent to that RTL.

5. **Observe circuit switching.** Reuse Week 4's CMOS gate/ngspice evidence and delay/capacitance evidence. The simulated voltages, currents, delays, and charging events prove the stated circuit and device models under the applied stimulus. They do not measure a physical cache, DRAM chip, or the energy of the selected C operation on a real processor.
6. **Close the chain with explicit boundaries.** Correlate the selected C data dependency with an architectural instruction effect, a matching teaching-model datapath event, synthesized logic of the same operation class, and transistor-level switching in a representative gate. Label every arrow as measured, simulated, decoded, modeled, synthesized, or inferred. The combined evidence supports a physically plausible causal account without claiming all artifacts are one implementation.

---

10. Systems consequences

Context switches and translation state

When the OS switches address spaces, the same virtual address may mean a different physical page. The kernel and architecture coordinate through a page-table-root register, ASIDs where available, and required invalidation or fencing operations. Stale translations are a correctness and isolation problem, not merely a performance problem.

Changing a PTE is not enough by itself

A translation may already be cached in a TLB. After changing page tables, the OS follows architecture-defined synchronization steps so later accesses do not keep using forbidden stale metadata. On multiple CPUs, this can require remote TLB invalidation—often called a TLB shootdown.

Page faults must be precise enough to resume

A demand-paged array can initially have no resident physical frame. The load faults; the kernel validates the virtual mapping, allocates or retrieves a page, updates page tables, performs required synchronization, and returns. The architectural load can then be retried. The cache and pipeline must not have committed the denied/incomplete load as if it succeeded.

Device memory is a different future path

Drivers may map device registers with cacheability and ordering attributes different from ordinary RAM. Using a normal cached-load mental model for MMIO can be incorrect. Later ToyOS and driver work will add memory types, barriers, interrupts, DMA, and IOMMU behavior deliberately.

---

11. Mastery: Explain · Draw · Observe · Build

Explain

Without notes:

- distinguish an I-cache miss, D-cache miss, TLB miss, and page fault;
- explain why a TLB entry contains translation and permission metadata rather than the loaded C value;

- explain why a cache hit cannot bypass a denied PTE;
- describe how a page-table walk creates memory traffic;
- explain why a stall changes timing but must preserve architectural state;
- assign page-table construction and fault response to OS policy;
- continue one `add` below the datapath into gates and physical switching.

Draw

Draw one request from virtual PC or load address through:

```
core → TLB → permission check → physical address → cache → lower memory
            ↘ fault state → privileged trap handler
```

Label architectural state, microarchitectural state, OS-owned data/policy, and physical realization separately. Add backpressure to the exact pipeline state that waits on each miss.

Observe

Preserve the Python output and annotate:

- one TLB hit;
- one TLB miss followed by a page-table walk;
- one cache miss/fill and later same-line hit;
- one permission fault despite a resident cache line;
- one absent-PTE fault;
- the modeled cycle cost and why it is not a hardware measurement.

Build

Extend the embedded model in one bounded way:

1. store actual line bytes and return `7` then `11` for the two array words;
2. add least-recently-used TLB replacement and print the victim;
3. implement a two-level page-table dictionary with explicit walk addresses;
4. emit an annotated two-iteration `sum` instruction trace;
5. add a dirty bit and safe writeback event without adding host file I/O.

Keep it deterministic, single-process, and confined to modeled addresses.

Why? notebook

1. Why can the instruction cache and data cache miss independently?
2. Why does a cache line use a physical identity?
3. Why does the page offset survive translation unchanged?
4. Why is a TLB miss often recoverable without an OS trap?
5. Why can a page fault support demand paging rather than indicate a bug?
6. Why must permission be checked before cached data is returned?
7. Why can a load-use dependency still stall despite forwarding hardware?
8. Why does changing a PTE require synchronization with cached translations?
9. Why does coherence not define the whole memory-ordering contract?

10. Why is “C executes on transistors” true but too imprecise to explain the responsible layers?

Final bridge to ToyOS

The core now has the mechanisms an operating system expects:

```
architectural state + privilege + precise traps
  + address translation + permission enforcement
  + interruptible, stallable memory access
```

ToyOS will supply the policy:

```
boot
  → install exception entry
  → allocate physical pages
  → construct page tables
  → map kernel and user addresses
  → handle faults
  → schedule task state
  → enter user mode and service system calls
```

The same four-layer discipline continues. RISC-V or x86-64 defines visible mechanisms; the CPU implements them; the kernel configures and responds to them; gates and transistors make every state transition physical.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Cache Tlb Trace

Format: Python lab

What you will do: Safe cache/TLB/MMU observation model with deterministic self-tests.

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
python3 05-riscv-cpu-microarchitecture/challenges/day-007-cache-tlb-trace.py
```

- View lab source on GitHub